台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第三篇

第 10 ～ 12 章

SLAM 數學基礎

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第三篇 SLAM 數學基礎

第 10 章

座標系統與剛體運動

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第二篇把感測器都介紹完了，但它們各說各話——光達說「前方三公尺有牆」，相機說「影像座標 (320, 240) 有個角點」，IMU 說「繞自己的 X 軸轉了 0.3 度」。這些話連單位與參考基準都不同，根本無法放在一起比較。

第三篇要做的，就是讓它們說同一種語言。而本章是第一步，也是最基礎的一步：建立座標系統，並學會把任何一筆資料從一個座標系搬到另一個座標系。

這是全書數學密度最高的一章，但它的回報也最大。從下一章開始，每一條式子裡都會出現本章的符號：第 11 章的狀態 x 就是本章的位姿，第 12 章要最佳化的變數就是一串位姿，第 13 章把光達點標進地圖用的就是本章的 2D 變換，第 17 章的 ICP 求的就是一個 T，第 22 章 ORB-SLAM2 追蹤的也是每一幀的 T。可以說，本章之後的所有章節，都是在求各式各樣的 T。

圖 10-0　第 10 章與全書的連結地圖

【承接前章】

第 4 至 9 章　六類感測器，各自用各自的座標與單位說話

本　　章　建立共同語言：座標系、位姿、變換矩陣

【本章負責】

座標系的三個層次：世界　→　機器人　→　感測器

齊次座標：把「旋轉 + 平移」寫成一次矩陣乘法

旋轉的三種表示法與它們的取捨（表 10-1）

TF Tree：ROS 如何管理整棵座標樹

【往下延伸】

→ 第 11 章　位姿成為待估計的「狀態 x」

→ 第 12 章　位姿圖的節點就是本章的 T

→ 第 13 章　光達點進地圖：本章 10.3 節公式的直接應用

→ 第 17 章　ICP 配準：在兩片點雲之間求一個 T

→ 第 28、29 章　TF 樹出錯是 ROS 建圖最常見的故障

學習目標

第二篇認識了機器人的所有感官，但每個感測器都在「自己的座標系」中回報資料——相機說「前方兩公尺有東西」、光達說「我的左邊三公尺有牆」。要把這些資訊拼成一張一致的地圖，必須先學會在不同座標系之間精確轉換。本章是第三篇的第一站，也是整個 SLAM 數學的地基。學完本章之後，同學應能夠達成下列目標：

1. 說明世界、機器人、感測器三種座標系的關係，並解釋為何 SLAM 需要多個座標系。
2. 使用齊次座標把旋轉與平移統一表示為單一矩陣運算。

3. 寫出 2D 位姿（x、y、θ）的表示法與座標轉換公式。
4. 說明 3D 旋轉的三種表示法——旋轉矩陣、歐拉角、四元數——及其優缺點與適用場合。
5. 說明座標轉換的組合與反轉，理解 TF 座標樹的概念。
6. 說明 ROS 中 map、odom、base_link、camera_link 等標準座標框架的意義與關係。

核心概念

中文名詞	英文名詞	概念說明
座標系	Coordinate Frame	描述位置與方向的參考基準，SLAM 中同時存在多個。
位姿	Pose	位置加姿態，2D 為（x、y、θ），3D 為位置加三軸旋轉。
剛體運動	Rigid Body Motion	不改變物體形狀與大小的運動，只含旋轉與平移。
齊次座標	Homogeneous Coordinates	多加一維的座標表示法，使旋轉與平移能以單一矩陣表示。
旋轉矩陣	Rotation Matrix	描述旋轉的正交矩陣，屬於特殊正交群 SO（3）。
歐拉角	Euler Angles	以滾轉、俯仰、偏航三個角度表示旋轉，直觀但有萬向鎖。
四元數	Quaternion	以四個數表示旋轉，無奇異點、運算高效，SLAM 的標準表示。
變換矩陣	Transformation Matrix	4×4 矩陣，同時包含旋轉 R 與平移 t，屬於 SE（3）。
萬向鎖	Gimbal Lock	歐拉角在特定姿態下喪失一個自由度的奇異現象。

中文名詞	英文名詞	概念說明
座標轉換樹	TF Tree	ROS 中管理各座標框架關係的樹狀結構。
里程計座標	odom frame	連續平滑但長期漂移的參考框架。
地圖座標	map frame	不漂移但可能跳變的全域參考框架。

10.1 世界座標、機器人座標、感測器座標

想像一台機器人在教室裡，一顆相機裝在它的頭部，拍到「正前方兩公尺處有一張椅子」。這句話裡藏著一個問題：「正前方」是相對於誰？相對於相機、相對於機器人的身體、還是相對於教室？這三者的答案都不一樣——機器人一轉身，相對相機的「前方」變了，但椅子相對教室的位置沒變。這就是為什麼 SLAM 必須同時處理多個座標系。

三個層次的座標系

- 世界座標系（World Frame）：固定不動的全域參考，通常以機器人開機的起點或地圖的某個角落為原點。地圖上所有東西的「絕對位置」都以它為基準。它是 SLAM 最終想把一切統一過去的座標系。
- 機器人座標系（Robot / Body Frame）：固定在機器人身上、隨機器人一起移動的座標系，原點通常設在底盤中心。在 ROS 中稱為 base_link。機器人的運動，本質上就是這個座標系相對世界座標系的移動。
- 感測器座標系（Sensor Frame）：固定在各感測器上。相機有相機座標系、光達有光達座標系，原點在各感測器的量測中心。感測器直接量到的資料，都在自己的座標系中。

為什麼需要分這麼多層

關鍵在於：感測器裝在機器人上的位置是固定的（相機永遠在頭部前方 20 公分、光達永遠在底盤上方 15 公分），但機器人相對世界的位置卻時刻在變。把「固定的安裝關係」與「變動的機器人位姿」分開處理，數學才會清晰。

於是一筆感測器資料要進地圖，需要走兩步轉換：先從感測器座標系轉到機器人座標系（用固定的安裝關係，稱為外參，第 5 章提過），再從機器人座標系轉到世界座標系（用機器人當前位姿，這正是 SLAM 要估計的）。理解這個「感測器 → 機器人 → 世界」的轉換鏈，是讀懂所有 SLAM 系統的前提。本章接下來就是要把「轉換」這件事的數學工具備齊。

10.2 齊次座標

座標轉換包含兩種動作：旋轉（轉個方向）與平移（移個位置）。麻煩的是，旋轉是「乘法」（矩陣乘向量）、平移是「加法」（向量加向量），兩者形式不同，串接多次轉換時會變得很囉嗦。齊次座標（Homogeneous Coordinates）是一個優雅的技巧，能把兩者統一成單一的矩陣乘法。

多加一維的魔法

做法很簡單：在原本的座標後面多補一個 1。一個 2D 點（x、y）寫成（x、y、1），一個 3D 點（x、y、z）寫成（x、y、z、1）。

這樣做的好處是，平移也能寫進矩陣裡了。以 2D 為例，「旋轉角度 θ 再平移（tx、ty）」可以合併成單一的 3×3 矩陣乘法：

$$[x';y';1] = [\cos\theta \;\; -\sin\theta \;\; tx;\sin\theta \;\; \cos\theta \;\; ty;0 \;\; 0 \;\; 1]\cdot[x;y;1]$$

這個 3×3 矩陣稱為 2D 的變換矩陣（或齊次變換矩陣）。左上角 2×2 是旋轉、右上角是平移、底下補一列（0 0 1）。原本要「先乘旋轉、再加平移」兩個步驟，現在一次矩陣乘法就完成。

為什麼這件事很重要

- 串接變換變成連乘：機器人經過多段移動，或資料經過「感測器→機器人→世界」多層轉換，只要把各段的變換矩陣依序相乘即可，不必分別處理旋轉與平移。
- 反向變換就是反矩陣：要把世界座標轉回機器人座標，只需求變換矩陣的反矩陣。
- 程式實作統一：所有座標處理都是矩陣乘法，程式碼簡潔、不易出錯，也便於 GPU 加速。

3D 的情況完全類似，只是變成 4×4 矩陣：左上角 3×3 是旋轉矩陣 R、右上角 3×1 是平移向量 t、底下補一列（0 0 0 1）。這個 4×4 矩陣是 SLAM 中最核心的數學物件之一，它同時裝著「機器人朝哪、在哪」的完整位姿資訊。

觀念釐清：變換矩陣的兩種讀法

同一個變換矩陣 T，可以有兩種解讀，初學者常混淆：

① 座標轉換：把「同一個點」在 A 座標系的座標，換算成在 B 座標系的座標。

② 剛體運動：把「一個點」實際移動、旋轉到新位置（在同一座標系中）。

兩者數學形式相同，但意義相反（把座標系轉一個方向，等於把點反向轉）。

SLAM 中大多是第①種——我們不是真的移動世界，而是換個座標系描述同一件事。

10.3 2D 位姿：x、y、θ

掃地機器人、教學小車都在平面上運動，只需 2D 位姿就足以描述。這是理解 3D 的最佳起點，因為 2D 只有一個旋轉自由度，一切都很直觀。

三個數描述一切

平面上一台機器人的位姿，用三個數就能完整描述：

- x、y：機器人在世界座標系中的位置。
- θ：機器人的朝向角（航向），即機器人的「正前方」與世界座標 X 軸的夾角。

這正是第 3 章航位推算、第 14 章 Gmapping 粒子所估計的量。每一個粒子、每一個位姿節點，在 2D SLAM 中就是這樣一組（x、y、θ）。

2D 座標轉換的實例

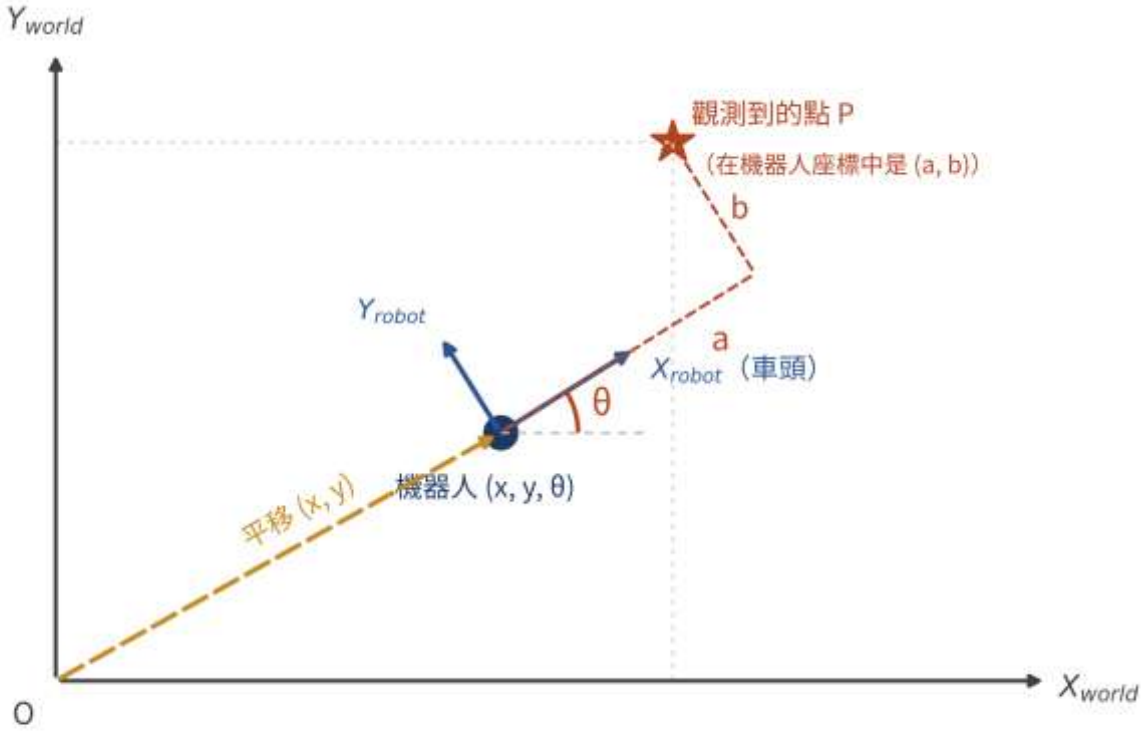
假設光達在機器人座標系中量到一個點 p_robot =（a、b）（機器人前方 a、左方 b），而機器人在世界座標系中的位姿是（x、y、θ），那麼這個點在世界座標系中的位置為：

$$x_world = x + a \cdot \cos\theta - b \cdot \sin\theta$$

$$y_world = y + a \cdot \sin\theta + b \cdot \cos\theta$$

這正是第 8 章「把一圈光達距離值畫進地圖」時，每個點都要做的計算。用齊次座標寫，就是前一節那個 3×3 變換矩陣乘上（a、b、1）。同學可以驗證：當 θ = 0（機器人朝正東）、位姿在原點時，公式退化為 x_world = a、y_world = b，符合直覺。

圖 10-1　2D 剛體變換：把機器人座標中的點搬到世界座標



$x_{world} = x + a\cos\theta - b\sin\theta$　　$y_{world} = y + a\sin\theta + b\cos\theta$

＝「先把點旋轉 θ，再平移 (x, y)」——這就是 2D 剛體變換

這張圖是全書使用頻率最高的一組式子的幾何來源。機器人位於世界座標的 (x, y)，車頭朝向與世界 X 軸夾角 θ。它用光達量到一個點，在自己的座標系中記為 (a, b)——a 是前方多遠、b 是左方多遠。問題是：這個點在世界地圖的哪裡？

答案分兩步。先旋轉：機器人的座標軸相對世界轉了 θ，所以 (a, b) 要先乘上旋轉矩陣，得到 (a·cosθ − b·sinθ, a·sinθ + b·cosθ)。再平移：加上機器人自己的位置 (x, y)。合起來就是圖中下方那兩條式子。

請務必記住順序是「先轉後移」而不能顛倒——這一點在第 17 章的 ICP 與第 22 章的位姿

更新中會反覆出現。顛倒順序會得到完全不同的結果，而且錯誤往往不明顯：機器人在原點附近時幾乎看不出來，一旦走遠就整張地圖歪掉。許多同學在第 29 章建圖時遇到「地圖隨著走動愈歪愈誇張」，根源就在這裡。

> **一個把觀念串起來的練習**
>
> 把這兩條式子倒過來想：如果已知點在世界的位置與機器人的位姿，要算它在機器人座標中的 (a, b)，該怎麼做？
>
> 答案是乘上反變換——而 10.6 節會告訴你，反變換不必真的求反矩陣，只要 $T^{-1} = [\, R^T \quad -R^T t \,]$。
>
> 這兩個方向在 SLAM 中都用得到：建圖時用正向（把觀測搬進地圖），定位時用反向（把地圖點搬進感測器視角去比對）。第 13 章與第 14 章正是這兩者的對照。

角度的兩個實務陷阱

- 角度環繞（Angle Wrapping）：θ 是週期性的，+180 度與 -180 度是同一個方向。計算兩個角度的差時，必須把結果規範回$(-\pi, \pi]$區間，否則會出現「轉了 350 度」這種本該是「反向轉 10 度」的荒謬結果。這是 2D SLAM 程式最常見的 bug 之一。
- 弧度與角度：數學運算一律用弧度（radian），但人類閱讀習慣用度（degree）。程式中務必統一單位，混用會造成難以察覺的錯誤。

10.4　3D 位姿：位置與姿態

無人機、機器狗、手持相機在三維空間中運動，需要完整的 3D 位姿。3D 位置很簡單，就是（x、y、z）三個數；真正複雜的是 3D 姿態——描述「朝哪」需要三個旋轉自由度，而表示這三個自由度的方法，竟有好幾種，各有優劣。這是本章的數學重點。

六個自由度

剛體在三維空間中共有六個自由度（6-DoF）：三個平移（沿 X、Y、Z 移動）加三個旋

轉（繞 X、Y、Z 轉動）。第 7 章、第 23 章反覆出現的「6-DoF 位姿」，指的就是這六個數。位置的三個自由度沒有爭議，以下三節專注於旋轉的三種表示法。

為什麼旋轉這麼麻煩？因為旋轉不是普通的量。兩個平移可以隨意相加、順序無關；但兩個旋轉的合成與順序有關——先繞 X 轉 90 度再繞 Y 轉 90 度，和先繞 Y 再繞 X，結果完全不同（同學可拿手機實際轉轉看）。這種「不可交換性」使得旋轉的數學遠比平移微妙，也是第 2 章 MIT VNAV 課程強調「流形上的最佳化」的原因——旋轉構成的空間是彎曲的，不是平坦的向量空間。

10.5 旋轉矩陣、歐拉角、四元數

這是本章最重要的一節。3D 旋轉有三種主要表示法，每一種都在 SLAM 系統中出現，同學必須理解它們各自的角色。

一、旋轉矩陣（Rotation Matrix）：理論的基石

用一個 3×3 的矩陣 R 表示旋轉。它把一個向量旋轉到新方向，運算就是矩陣乘向量。旋轉矩陣有嚴格的數學性質：它是正交矩陣（各列互相垂直且長度為 1）、行列式為 +1，這類矩陣構成的集合稱為特殊正交群 SO（3）。

- 優點：數學性質完美、直接可用於座標轉換、無奇異點、旋轉合成就是矩陣相乘。
- 缺點：用 9 個數表示只有 3 個自由度的旋轉，冗餘度高；儲存與最佳化時，還必須維持「正交」這個約束，不能自由地對 9 個數做加減（否則就不再是合法旋轉）。

二、歐拉角（Euler Angles）：最直觀但有陷阱

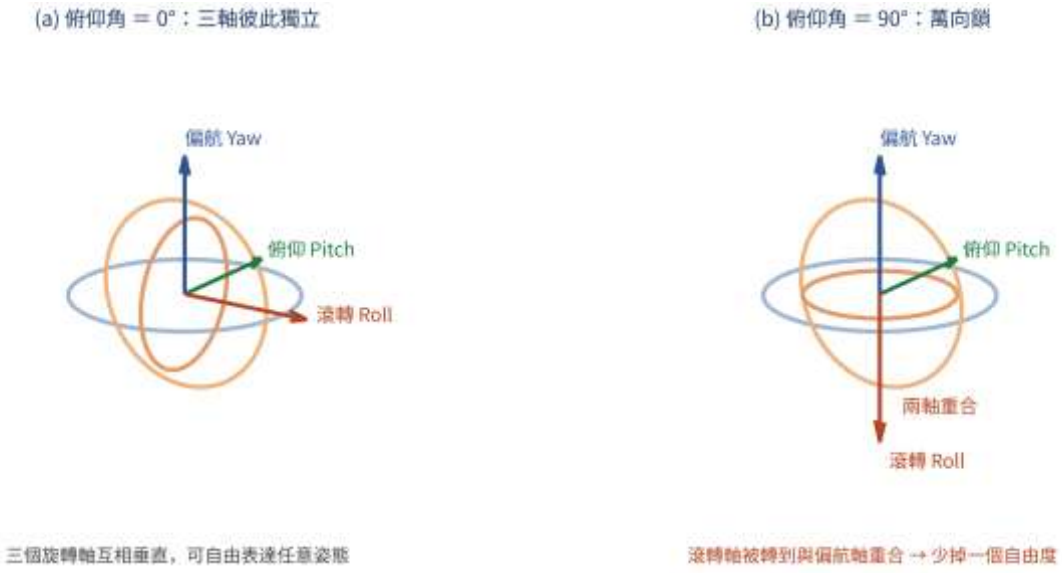
用三個角度——滾轉 φ、俯仰 θ、偏航 ψ（第 7 章介紹過）——表示旋轉。這是最貼近人類直覺的方式，「機頭上仰 10 度、右傾 5 度、朝向東方」一聽就懂。

- 優點：直觀、只用 3 個數（無冗餘）、便於人類理解與介面顯示。

- 缺點：有萬向鎖（Gimbal Lock）——當俯仰角接近 ±90 度時，滾轉與偏航的旋轉軸重合，喪失一個自由度（第 7 章詳述）；此外，旋轉合成不能直接相加，且同一旋轉可能有多組角度表示（週期性），數值最佳化時容易出問題。

因此歐拉角適合「給人看」（介面顯示、除錯），但不適合「給機器算」（內部計算）。這是一個重要的工程分工。

圖 10-2　萬向鎖：歐拉角為什麼會失去一個自由度



萬向鎖是歐拉角最惡名昭彰的缺陷，但它其實不難理解——只要看到那三個軸的關係就夠了。左圖中俯仰角為零，偏航、俯仰、滾轉三個旋轉軸互相垂直，任意姿態都能以三個角度唯一表達。

右圖把俯仰角轉到 90 度。注意發生了什麼事：滾轉軸被俯仰軸帶著一起轉，轉到與偏航軸重合了。於是「改變偏航角」與「改變滾轉角」產生完全相同的運動——三個參數只剩下兩個有效的自由度，姿態空間中有一整個方向變得無法表達。

這不是實作上的瑕疵，而是數學上的必然：任何用三個數描述三維旋轉的方法，都一定存在奇異點。想避開它，就必須用超過三個數——旋轉矩陣用九個（帶六個約束），四元數用四個（帶一個約束）。這就是 10.5 節為什麼說四元數是 SLAM 的實戰標準。

案例 10-1 的阿波羅 11 號導航平台正是栽在這件事上。而在機器人身上，它一樣會出事：無

人機做大角度俯衝或爬升、機械手臂的關節轉到特定姿態、四足機器人翻倒時，只要姿態以歐拉角儲存，就可能出現數值爆炸或姿態跳變。第 26 章討論無人機飛控時會再遇到它。

三、四元數（Quaternion）：SLAM 的實戰標準

四元數用四個數（w、x、y、z）表示旋轉，可以理解為「繞某個軸轉某個角度」的一種緊湊編碼。它初看不直觀，卻是現代 SLAM、無人機、遊戲、動畫內部一致採用的旋轉表示。

- 優點：無萬向鎖（沒有奇異點）、只用 4 個數（冗餘極小）、旋轉合成只需四元數乘法（運算高效）、數值穩定、便於插值（平滑地從一個姿態轉到另一個）。
- 缺點：不直觀，人難以直接讀懂（w、x、y、z）代表什麼方向；需要維持「單位長度」的約束（但只有一個約束，遠比旋轉矩陣的正交約束簡單）。

實務上的分工非常清楚：內部計算與儲存用四元數、介面顯示與人工判讀轉成歐拉角、座標轉換運算轉成旋轉矩陣。三者可以互相轉換，ROS 的 tf2 與各種數學函式庫都內建轉換函式。同學不必背誦四元數的乘法規則，但必須理解「為什麼實務都用它」——答案就是：它同時避開了旋轉矩陣的冗餘與歐拉角的萬向鎖。

表示法	數字個數	奇異點	主要優點	主要用途
旋轉矩陣	9(3×3)	無	座標轉換直接、理論完美	座標變換運算
歐拉角	3	有（萬向鎖）	最直觀、便於人讀	介面顯示、除錯
四元數	4	無	緊湊、高效、便於插值	內部計算與儲存

表 10-1　三種 3D 旋轉表示法比較

10.6　座標轉換與 TF Tree

有了變換矩陣這個工具，現在來看真實機器人系統如何管理眾多座標系之間的關係。答案是一個樹狀結構——TF 座標樹。

變換的組合與反轉

先複習兩個關鍵運算，它們是 TF 樹運作的基礎：

- 組合（Composition）：若已知「A 到 B 的變換」與「B 到 C 的變換」，把兩個變換矩陣相乘，就得到「A 到 C 的變換」。這讓我們能把感測器資料一路轉換到世界座標。
- 反轉（Inverse）：若已知「A 到 B 的變換」，求它的反矩陣，就得到「B 到 A 的變換」。

有了這兩個運算，只要知道任意兩個相鄰座標系之間的變換，就能算出任意兩個座標系之間的關係——這正是 TF 樹的核心思想。

TF Tree：座標系的家譜

在一台真實機器人上，座標系可能有十幾個：世界、地圖、里程計、底盤、每個輪子、光達、相機、機械臂的每一節……ROS 用一個樹狀結構（TF Tree，座標轉換樹）來管理它們的關係。

- 每個座標系是樹上的一個節點。
- 每個父子節點之間，記錄著一個變換（父到子）。
- 查詢任意兩個座標系的關係時，系統自動沿著樹路徑把沿途的變換組合（相乘）起來。

這個設計的巧妙之處在於：每個變換只需由「最了解它的那個模組」負責發布。光達驅動發布「底盤到光達」的固定變換（來自安裝量測）、SLAM 系統發布「地圖到底盤」的變換（來自定位估計）、里程計發布「里程計到底盤」的變換。各模組各司其職，TF 樹把它們自動整合成完整的座標關係網。第 29 章實作時，同學會親手設定這棵樹。

時間戳記：一個關鍵細節

TF 不只管「空間關係」，還管「時間」。因為機器人在動，每個變換都是隨時間變化

的，TF 系統為每個變換都標上時間戳記，查詢時可以問「在 0.1 秒前那一刻，相機相對地圖在哪」。這對感測器融合至關重要——相機與光達的資料到達時間不同，必須用正確時刻的變換來對齊，否則高速運動時會產生明顯誤差。這呼應了第 5 章融合中「時間同步」的工程課題。

10.7　ROS 中的 map、odom、base_link、camera_link

ROS 為行動機器人定義了一套標準座標框架命名與階層，幾乎所有 ROS 導航系統都遵循它。理解這套慣例，是讀懂與實作 SLAM 系統的基本功。

標準框架階層

從上到下的標準鏈是：map → odom → base_link → 各感測器框架。

圖 10-3　ROS 標準座標框架階層

map（地圖座標系）：全域固定、不漂移，但會跳變

↓　（由 SLAM / 定位系統發布）

odom（里程計座標系）：連續平滑，但長期漂移

↓　（由里程計發布）

base_link（機器人本體）：固定在機器人底盤中心

↓　（由固定安裝關係發布，即外參）

laser / camera_link / imu_link（各感測器座標系）

為什麼要 map 和 odom 兩層

這是整套設計最精妙、也最容易困惑的地方。為什麼不直接用一個座標系，要分成 map 與 odom 兩層？因為它們各自解決不同的需求，而且是互補的：

- odom（里程計座標系）：由里程計與 IMU 積分得到。它的特性是連續、平滑、不跳變——非常適合短期的運動控制與避障。但如第 3、7 章所述，它會長期漂移，越走越不準。
- map（地圖座標系）：由 SLAM 或定位系統（如 AMCL）得到。它的特性是長期正確、不漂移——因為它靠比對地圖來校正。但它會跳變：當 SLAM 突然發現「我其實在這裡」而修正位置時，機器人在 map 中的座標會瞬間跳動一下。

兩層並存的智慧在於：讓需要「平滑」的模組（如避障控制）用 odom，讓需要「準確」的模組（如全域路徑規劃）用 map。SLAM 系統實際發布的，是「map 到 odom」的變換——它不斷微調這個變換來補償里程計的漂移，而 base_link 相對 odom 的運動仍然平滑。這個設計讓「平滑」與「準確」兩個看似衝突的需求，優雅地共存。

感測器框架與外參

base_link 之下的每個感測器框架（laser、camera_link、imu_link 等），相對 base_link 的變換是固定的，由感測器在機器人上的安裝位置決定，也就是第 5 章提過的外參。這些變換通常寫在機器人的 URDF 描述檔中（第 29 章），或由靜態變換發布器發布。準確量測這些外參，是系統精度的基礎——安裝位置差幾公分或幾度，建出的地圖就會模糊或錯位。

至此，同學已經備齊了 SLAM 的第一組數學工具：如何在座標系之間精確地轉換位姿。但真實世界的量測都帶著雜訊——里程計會滑、光達有誤差、相機會誤匹配。下一章「機率、雜訊與狀態估測」，將教我們如何用機率的語言描述這些不確定性，並在其中做出最合理的估計。這是 SLAM 從「幾何」邁向「估測」的關鍵轉折。

公式或流程

一、3D 齊次變換矩陣

3D 剛體運動以 4×4 齊次變換矩陣 T 表示，同時包含旋轉 R（3×3）與平移 t(3×1):

$$T = [R\ t; 0\ 1]\quad (左上 3\times3 為 R，右上 3\times1 為 t，左下為 0，右下為 1)$$

把 A 座標系中的點 p_A（以齊次座標表示）轉到 B 座標系：

$$p_B = T_BA \cdot p_A$$

這裡 T_BA 讀作「從 A 到 B 的變換」。下標的書寫順序是 SLAM 中最容易出錯的地方，建議固定一種慣例並貫徹到底。

二、變換的組合與反轉

組合：把 A→B 與 B→C 的變換串接成 A→C:

$$T_CA = T_CB \cdot T_BA$$

注意相乘順序：相鄰的下標要對齊（B 與 B 相鄰）。反轉：

$$T_AB = (T_BA)^{-1}$$

齊次變換矩陣的反矩陣有簡便公式，不需做一般的 4×4 反矩陣運算：

$$T^{-1} = [R^T\ \ -R^T t; 0\ 1]$$

利用旋轉矩陣的正交性（反矩陣等於轉置），反轉變得非常高效。

三、2D 位姿複合（里程計串接）

機器人從位姿 $p_1 = (x_1, y_1, \theta_1)$ 經一段相對運動（$\Delta x, \Delta y, \Delta\theta$，在 p_1 的機體座標系中）後，新位姿為：

$$x_2 = x_1 + \Delta x \cdot \cos\theta_1 - \Delta y \cdot \sin\theta_1$$

$$y_2 = y_1 + \Delta x \cdot \sin\theta_1 + \Delta y \cdot \cos\theta_1$$

$$\theta_2 = \text{normalize}(\theta_1 + \Delta\theta)\quad (\text{normalize 表示規範回 } (-\pi, \pi])$$

這是第 3 章航位推算、第 14 章 Gmapping 運動更新的具體實現。normalize 步驟即 10.3 節強調的角度環繞處理，不可省略。

四、感測器資料進地圖的完整流程

圖 10-4　一筆感測器量測進入世界地圖的轉換鏈

① 感測器量測：點 p 在感測器座標系（如 camera_link）

↓ × T（base_link ← camera_link）　外參，固定

② 點 p 在機器人座標系（base_link）

↓ × T（odom ← base_link）　里程計

③ 點 p 在里程計座標系（odom）

↓ × T（map ← odom）　SLAM 修正

④ 點 p 在世界地圖座標系（map）→ 寫入地圖

整條鏈就是多個變換矩陣的連乘，TF 樹自動完成這個計算。

10.8　旋轉表示法的當代選擇：從工程到學習

10.5 節比較了旋轉矩陣、歐拉角與四元數。這個比較在傳統 SLAM 的脈絡下已經很完整，但近年有兩個發展值得補充——一個關於工程實作，一個關於深度學習。

一、量化的比較：各表示法的真實代價

近年有研究對七種主要的三維旋轉表示法做了系統性的比較，涵蓋歐拉角、軸角向量、四元數、旋轉矩陣、指數映射、六維連續表示法，以及矩陣 Fisher 與 Bingham 等機率式表示法，並從連續性、萬向鎖、運算效率、儲存量與內插品質等面向評估。

其結論與本書 10.5 節一致：就一般用途而言，四元數取得了最佳的整體平衡——運算效率高、儲存精簡、可用球面線性內插（SLERP）平滑過渡，且沒有萬向鎖。具體數字上，四元數的複合運算約 34 微秒、佔用 32 位元組；旋轉矩陣約 306 微秒、72 位元組；六維連續表

示法約 422 微秒、48 位元組。可見四元數在複合運算上快了近一個數量級，這在每秒要做數萬次位姿複合的 SLAM 後端中是實質的差異。

二、但在神經網路裡，四元數不是最好的

第二個發展與第 32 章關係密切。當機器人改用神經網路直接預測姿態時——例如從影像回歸相機位姿、或讓視覺語言動作模型輸出機械手的目標姿態——四元數反而成了問題。

原因是連續性。神經網路以梯度下降訓練，要求輸出空間對輸入的變化是連續的；但四元數有「雙重覆蓋」的性質（q 與 -q 代表同一個旋轉），歐拉角更有萬向鎖的跳變。這些不連續處會讓網路在訓練時收到互相矛盾的梯度，學不穩定。

解法是改用六維連續表示法：輸出旋轉矩陣的前兩個欄向量，再以施密特正交化還原出完整的旋轉矩陣。這個表示法在旋轉空間上是連續的，沒有任何奇異點。上述研究指出，在神經網路的姿態估測任務中，它相較四元數可降低約百分之六至十四的誤差。

這帶出一個對同學很重要的觀念：沒有「最好的旋轉表示法」，只有「對這個任務最好的表示法」。SLAM 後端要快速複合，所以用四元數；神經網路要連續可微，所以用六維表示；人要看得懂，所以在使用者介面上顯示歐拉角。同一個旋轉，依用途換不同的衣服。

三、李群：讓最佳化在流形上正確進行

還有一條在現代 SLAM 中已成標準、但入門教材常略過的路線：把旋轉與位姿視為李群（SO(3) 與 SE(3)），把它們的微小變化視為李代數上的向量。

為什麼需要這個？因為第 12 章的最佳化要對位姿求導數、要算「兩個位姿相差多少」、要做「位姿加上一個微小增量」。但旋轉矩陣不能直接相加——兩個旋轉矩陣相加之後不再是旋轉矩陣。李代數提供了正確的做法：在流形的切空間上做加減與求導，再用指數映射送回流形。

實務上同學不必從頭推導——Sophus、manif 這類函式庫已經把 SO(3)、SE(3) 的運算封裝好，g2o、Ceres、GTSAM 等最佳化框架也都內建了流形上的參數化。第 12 章與第 23 章

會再具體用到。這裡只要先記住一句話：旋轉不是向量，不能直接加。本章 10.3 節提醒的「角度不能直接平均」，正是這句話在二維的簡化版。

一張選用速查表

使用者介面、規格書、人要讀的數字　→　歐拉角（但內部絕不以它儲存）

SLAM 內部狀態、感測器驅動、ROS 訊息　→　四元數

最佳化的求導與增量更新　→　李代數（so(3) / se(3)）

神經網路的輸出層　→　六維連續表示法

點雲與幾何運算　→　旋轉矩陣（直接可乘）

案例

案例 10-1　阿波羅 11 號的萬向鎖驚魂

萬向鎖不是抽象的數學名詞，它曾在人類登月任務中造成真實危機。阿波羅太空船的慣性導航平台用三個機械環（萬向架）支撐，以歐拉角的方式追蹤太空船姿態。當三個環接近共面時，就會發生萬向鎖——喪失一個旋轉自由度，導航平台可能失去姿態基準。

工程師深知這個風險，在儀表板上設計了警示，並規劃飛行姿態盡量避開危險區域。太空人麥可·柯林斯曾半開玩笑地抱怨這個限制。事後來看，這正是「用歐拉角表示姿態」的先天缺陷在極端任務中的體現。現代太空船、無人機、機器人早已改用四元數，正是為了根除這個問題。

這個案例給同學的啟示是：表示法的選擇不是純粹的數學品味，它有真實的工程後果。10.5 節看似枯燥的三種旋轉表示法比較，背後是無數工程師用教訓換來的智慧——這也是為什麼本書堅持在數學章節中講清楚「為什麼這樣選」，而不只是「怎麼算」。

案例 10-2　RViz 裡的座標樹：讓抽象變具體

第一次接觸 TF 樹的同學，常覺得它抽象難懂。但在 ROS 的視覺化工具 RViz 中，座標樹會變成看得見的東西：每個座標系顯示為一組紅綠藍的小軸（對應 X、Y、Z），當機器人移動，base_link 的軸就跟著動；當光達安裝在機器人前方，laser 的軸就固定顯示在 base_link 前方一小段距離。

更有教育意義的是「觀察錯誤」。如果外參設錯了（例如把光達的安裝方向填反），RViz 中光達掃描的點雲就會與實際環境「錯位」——牆明明在前方，點雲卻畫在後方或側面。同學透過調整外參、即時觀察點雲是否貼合環境，能非常直觀地理解座標轉換的意義。這是把 10.1 到 10.7 節所有抽象概念一次串起來的最佳實驗，第 29 章實作會大量用到。

大學課堂上一個常見的除錯口訣是：「地圖歪了先看 TF」。許多 SLAM 建圖失敗，根源不在演算法，而在座標框架設定錯誤——外參填錯、時間戳記沒對齊、TF 樹斷裂。理解本章，能幫同學省下無數除錯的冤枉路。

案例 10-3　機械手臂的正向運動學：同一套數學的另一個舞台

本章的座標變換數學，不只用於行動機器人，也是機械手臂的核心。一支六軸機械手臂，從基座到夾爪之間有六個關節，每個關節的轉動都是一個旋轉變換。把基座到第一節、第一節到第二節……一路到夾爪的變換矩陣依序相乘，就能算出「給定六個關節角度，夾爪在空間中的位姿」——這稱為正向運動學（Forward Kinematics）。

這正是 10.6 節「變換組合就是矩陣連乘」的直接應用。同學若在大學同時修習機械手臂相關課程，會發現兩門課用的是同一套齊次變換數學——這並非巧合，而是因為「剛體在空間中如何運動」是機器人學共通的語言。學好本章，等於同時打下了行動機器人與機械手臂的數學基礎。這也呼應第 2 章各校課程「感測器→數學→系統」的共同架構：數學是所有機器人技術的公分母。

實作活動

實作 10-1 座標轉換動手算與 TF 樹觀察

目的：透過紙筆計算與程式驗證，掌握 2D 座標轉換；並在 ROS 中觀察真實的 TF 樹，把抽象概念變具體。本實作分紙筆、程式、ROS 三部分，適合不同深度的教學。

第一部分：紙筆計算（全體）

1. 一台機器人在世界座標系中位姿為（x=3, y=2, θ=90°）。它的光達在機體座標系中量到一個點（前方 a=2，左方 b=0）。用 10.3 節公式手算這個點的世界座標，並在方格紙上畫圖驗證答案合理。
2. 同一台機器人，光達量到另一點（a=1, b=1）。手算其世界座標。畫圖確認。
3. 角度環繞練習：機器人朝向 θ_1=170°，右轉 Δθ=30°。直接相加得 200°，把它規範回(-180°, 180°]應是多少？說明為何不處理會出問題。

第二部分：Python 驗證（全體）

4. 用 numpy 建立 10.2 節的 3×3 齊次變換矩陣，把第一部分的計算改用矩陣乘法完成，驗證結果一致。
5. 實作變換的組合：給定「世界→機器人」與「機器人→光達」兩個變換，相乘得到「世界→光達」，並驗證。
6. 實作反轉：求「世界→機器人」變換的反矩陣，驗證它等於「機器人→世界」變換（用 10.公式的簡便法與 numpy 直接求逆兩種方式比較）。
7. （大學部）實作歐拉角、旋轉矩陣、四元數三者的互相轉換（可用 scipy.spatial.transform.Rotation），取一個俯仰角接近 90° 的姿態，觀察歐拉角表示如何出現異常，親身驗證萬向鎖。

第三部分：ROS TF 樹觀察（有 ROS 環境者）

8. 啟動 TurtleBot3 模擬（第 28 章）或任一含 TF 的 ROS 系統，用 view_frames 或 rqt_tf_tree 工具產生 TF 樹的圖，找出 map、odom、base_link 與各感測器框架的階層關係。

9. 在 RViz 中顯示 TF，移動機器人，觀察哪些座標軸跟著動（base_link 及其子框架）、哪些不動（map）。
10. 用 tf2 的查詢工具，查詢「camera_link 相對 map」的變換，並在機器人移動前後各查一次，觀察數值變化。

討論問題

1. 第一部分中，機器人朝向 90°（正北）時，它機體的「前方」對應世界座標的哪個方向？你的計算結果符合直覺嗎？
2. 為什麼 SLAM 系統發布的是「map→odom」變換，而不是直接發布「map→base_link」？這樣設計對「平滑」與「準確」的兼顧有何幫助？
3. （大學部）你在萬向鎖實驗中觀察到什麼異常？如果無人機的飛控用歐拉角而非四元數，在做翻滾特技時可能發生什麼事？
4. 若一台機器人建出的地圖總是「歪斜」或「牆的點雲對不上」，依本章知識，你會優先檢查哪些地方？

延伸挑戰（大學部）

撰寫一個 ROS 2 節點，發布一個自訂感測器相對 base_link 的靜態變換（故意設一個已知的旋轉與平移），在 RViz 中顯示該感測器的模擬點雲，調整外參數值並即時觀察點雲位置變化，直到點雲貼合預期位置。這個練習直接對應第 29 章設定機器人 TF 的實務，能為 2D SLAM 小車實作打下座標基礎。

延伸閱讀

1. 高翔、張濤，《視覺 SLAM 十四講》第 3 講（三維空間剛體運動）、第 4 講（李群與李代數），電子工業出版社.（華文世界最清楚的 SLAM 旋轉數學入門）

2. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017.（剛體運動與座標變換的權威教科書，有免費電子版與線上課程）
3. J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," arXiv, 2018.（旋轉群與流形最佳化的友善入門，銜接第 12 章）
4. ROS 官方文件之 tf2 教學（docs.ros.org）.（TF 座標樹的實作指引，附完整範例）
5. REP 105: Coordinate Frames for Mobile Platforms(ros.org/reps).（ROS 標準座標框架 map / odom / base_link 的官方定義文件）
6. 3Blue1Brown, "Quaternions and 3D rotation" 系列影片（YouTube）.（以絕佳動畫直觀解釋四元數，強烈推薦）
7. R. Siegwart et al., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011.（第 2、3 章含座標系與運動學的機器人學脈絡）

從確定到不確定：本章的位置回顧

本章做完了一件事：讓所有感測器的資料能表達在同一個座標系中。但它悄悄假設了一個在真實世界裡並不成立的前提——每一個數字都是精確的。

圖 10-1 的式子假設機器人的位姿 (x, y, θ) 是已知且正確的。但第 7 章告訴我們 IMU 會漂移，第 5 章告訴我們 GNSS 有 DOP 放大的誤差，第 13 章的輪式里程計會打滑。實際上機器人從來不確切知道自己在哪裡——它只有一個帶著不確定度的估計。

同樣地，圖 10-1 中「觀測到的點 P」也不是一個精確的點。第 8 章的光達有測距誤差與角解析度限制，第 9 章的相機有像素量化與特徵點定位誤差。每一筆觀測都是一團模糊。

於是第 11 章要把本章所有的等號換成機率分布：位姿不再是三個數，而是一個高斯分布；觀測不再是一個點，而是帶著共變異數的量測。而融合多個感測器的工作，就變成「把多個機率分布乘在一起，找出最可能的那個答案」。這就是貝氏濾波，也是卡爾曼濾波與粒子濾

波的共同骨架。本章的變換矩陣不會消失——它會成為機率模型中的運動模型與觀測模型。

小結

1. SLAM 同時處理世界、機器人、感測器三層座標系；感測器安裝關係固定（外參）、機器人位姿變動（SLAM 要估的量），一筆資料進地圖需經「感測器→機器人→世界」的轉換鏈。
2. 齊次座標多補一維，把旋轉（乘法）與平移（加法）統一為單一矩陣乘法；2D 用 3×3、3D 用 4×4 變換矩陣，使串接變換成為連乘、反轉成為求反矩陣。
3. 2D 位姿以（x、y、θ）三個數描述，座標轉換用含 cosθ、sinθ 的公式；實務上務必處理角度環繞（規範回 -π 到 π）並統一弧度與角度單位。
4. 3D 剛體有六個自由度（三平移三旋轉）；旋轉的合成不可交換，構成的空間是彎曲的，這是旋轉數學複雜與「流形最佳化」的根源。
5. 3D 旋轉三種表示法各有分工：旋轉矩陣（9 數、無奇異、用於座標轉換）、歐拉角（3 數、直觀但有萬向鎖、用於人機介面）、四元數（4 數、無奇異、高效、用於內部計算與儲存）。
6. TF 樹以樹狀結構管理眾多座標系，每個變換由最了解它的模組發布，查詢時自動沿路徑連乘；變換附時間戳記以支援感測器融合的時間對齊。
7. ROS 標準框架 map→odom→base_link→感測器：odom 連續平滑但漂移、map 準確但跳變；SLAM 發布「map→odom」變換來補償漂移，讓「平滑」與「準確」兩需求優雅共存。

本章備齊了 SLAM 的第一組數學工具——確定性的座標轉換。但真實世界充滿不確定性：里程計會滑、光達有雜點、相機會誤匹配。下一章「機率、雜訊與狀態估測」將引入機率的語言，教機器人如何描述「我大概在這裡，但不太確定」，並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵一步，也是貝氏濾波、卡爾曼濾波與粒子濾波的共同

起點。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 11 章

機率、雜訊與狀態估測

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第 10 章讓所有感測器的資料能表達在同一個座標系中，但它假設每個數字都是精確的。本章要拆掉這個假設。

真實的機器人從來不確切知道自己在哪裡。輪子會打滑、IMU 會漂移、光達有測距誤差、相機的特徵點會定位不準。更麻煩的是，這些誤差還會互相矛盾——里程計說走了三公尺，光達比對說只走了二點七公尺，到底該信誰？

本章給的答案是：都信，但依照各自的可信度加權。而「可信度」要能被計算，就必須先把每一個估計都寫成機率分布。於是位姿不再是三個數，而是一個高斯分布；觀測不再是一個點，而是帶著共變異數的量測；融合不再是取平均，而是把分布相乘。

這是全書觀念上最關鍵的轉折。第 12 章的最佳化、第 14 章的 Gmapping、第 15 章的 Cartographer、第 23 章的 ORB-SLAM3，全部建立在本章的框架之上——它們的差別只在於用什麼方式表示那個分布、用什麼方式求出最可能的答案。

圖 11-0 第 11 章與全書的連結地圖

【承接前章】

第 10 章 位姿 = 三個（或六個）精確的數字

本 章 位姿 = 一個帶著不確定度的機率分布

【本章負責】

高斯分布與共變異數：用均值 + 橢圓描述「我大概在哪」

運動模型與觀測模型：預測與修正的兩個來源

貝氏濾波 → 卡爾曼 → EKF → 粒子濾波

【往下延伸】

→ 第 12 章 同一個問題的另一種解法：最佳化（濾波的對照組）

→ 第 13 章 佔據格地圖的每一格也是一個機率

→ 第 14 章 Gmapping = 粒子濾波 + 每顆粒子帶一張地圖

→ 第 16 章 AMCL：掃地機器人的重定位就是本章的 MCL

→ 第 23 章 ORB-SLAM3 的 MAP 估測是本章觀念的最佳化版

學習目標

上一章我們學會了在座標系之間精確轉換——那是「確定性」的數學。但真實世界沒有這麼乾淨：里程計會打滑、光達有雜點、相機會誤匹配。本章引入機率的語言，教機器人如何說出「我大概在這裡，但不太確定」，並在雜訊中做出最合理的估計。這是 SLAM 從幾何邁向估測的關鍵轉折，也是全書數學的核心。學完本章之後，同學應能夠達成下列目標：

7. 說明 SLAM 為何必須以機率方式處理不確定性，而非追求單一精確答案。
8. 解釋高斯分布的均值與變異數的意義，並理解共變異數如何描述二維以上的不確定性。
9. 說明運動模型與觀測模型如何以機率描述機器人的預測與感知。

10. 說明貝氏濾波「預測—更新」兩步驟的遞迴架構，理解它是所有濾波器的共同骨架。

11. 說明卡爾曼濾波器的運作，並解釋卡爾曼增益如何權衡預測與觀測。

12. 說明 EKF 如何處理非線性，以及 EKF-SLAM 的基本概念與限制。

13. 說明粒子濾波與蒙地卡羅定位的原理，並比較它與卡爾曼家族的差異。

核心概念

中文名詞	英文名詞	概念說明
機率分布	Probability Distribution	描述隨機變數各種可能取值的機率。
高斯分布	Gaussian Distribution	鐘形的常態分布，以均值與變異數完整描述。
均值	Mean	分布的中心，代表最可能的估計值。
變異數	Variance	分布的離散程度，代表不確定性的大小。
共變異數	Covariance	描述多維隨機變數之間的相關性與聯合不確定性。
運動模型	Motion Model	描述機器人執行控制後，狀態如何變化（含雜訊）。
觀測模型	Observation Model	描述在某狀態下，感測器會量到什麼（含雜訊）。
貝氏濾波	Bayes Filter	以預測與更新兩步驟遞迴估計狀態的通用架構。
卡爾曼濾波	Kalman Filter	線性高斯系統下的最佳遞迴估測器。
卡爾曼增益	Kalman Gain	決定相信預測還是觀測的權重。
擴展卡爾曼濾波	EKF	將非線性系統線性化後套用卡爾曼濾波。
粒子濾波	Particle Filter	以大量樣本近似任意分布的非參數濾波器。
蒙地卡羅定位	MCL	以粒子濾波在已知地圖中定位的方法，即 AMCL。

11.1 為什麼 SLAM 一定要處理不確定性

回想第 7 章那個令人印象深刻的實驗：一台完全靜止的裝置，靠 IMU 積分算出的「位置」在 60 秒內漂移了數公尺。再想第 4 章的 Wi-Fi 定位、第 5 章的 GPS 跳點、第 9 章相機的誤匹配——沒有任何一個感測器給的是「真值」，它們給的都是「帶著雜訊的猜測」。

面對這個現實，有兩種態度。天真的做法是：假裝感測器是準的，直接用它的讀值。這正是航位推算失敗的原因——把每個帶誤差的量測當成真值累加，誤差就永無止境地累積。

成熟的做法是：承認每個量測都不確定，並量化這個不確定性，然後在多個不確定的資訊之間做最佳的權衡。這就是機率機器人學（Probabilistic Robotics）的核心思想，也是現代 SLAM 的數學骨幹。

關鍵觀念：估計的是「分布」而非「數值」

這是全章最重要的觀念轉變。傳統思維認為「機器人在（3.5, 2.1）」；機率思維則認為「機器人最可能在（3.5, 2.1）附近，但也可能在周圍一個橢圓範圍內，越靠近中心可能性越高」。

換句話說，SLAM 估計的不是一個點，而是一整個機率分布——它同時告訴我們「最佳猜測」與「這個猜測有多可信」。這個「有多可信」的資訊至關重要，因為它決定了下一步該多相信這個估計。第 1 章給出的 SLAM 機率定義 $p(x, m \mid z, u)$，正是在說「給定所有觀測與控制，狀態的機率分布長什麼樣」。

直觀比喻：蒙眼走路的人

想像你蒙著眼在熟悉的房間走動。每走一步，你「大概」知道自己前進了多少（運動），

但不太確定——可能多一點少一點（運動雜訊）。這時你對自己位置的把握越來越

模糊。

突然你的手摸到了牆角（觀測）。這個明確的地標，讓你對位置的把握瞬間變清晰。

SLAM 就是在做這件事：走路時不確定性擴大（預測），摸到地標時不確定性收縮（更新）。

本章要學的，就是如何用數學精確地描述「模糊」與「變清晰」這兩個動作。

11.2 高斯分布、均值、變異數、共變異數

要用數學描述不確定性，首先需要一種「分布」。SLAM 中最常用的是高斯分布（Gaussian Distribution，又稱常態分布），因為它有兩個無可取代的優點：只需兩個參數就能完整描述、而且在各種運算下仍保持高斯形式（數學上極為友善）。

一維高斯：鐘形曲線

一維高斯分布就是熟悉的鐘形曲線，由兩個參數完整決定：

- 均值 μ(mu)：曲線的中心，代表「最可能的值」，也就是我們的最佳估計。
- 變異數 σ^2（sigma 平方）：曲線的胖瘦，代表不確定性的大小。變異數小，曲線又高又窄，表示很有把握；變異數大，曲線又矮又寬，表示很不確定。

標準差 σ 是變異數的平方根，與量測值同單位，更直觀。「機器人在 3.5 公尺處，標準差 0.2 公尺」意思是：最佳估計是 3.5，而真值有約 68% 的機率落在 3.3 到 3.7 之間（高斯分布的「一個標準差」範圍）。

多維高斯：從點到橢圓

機器人的位置是二維的（x、y），不確定性也就變成二維。此時均值是一個點（最可能

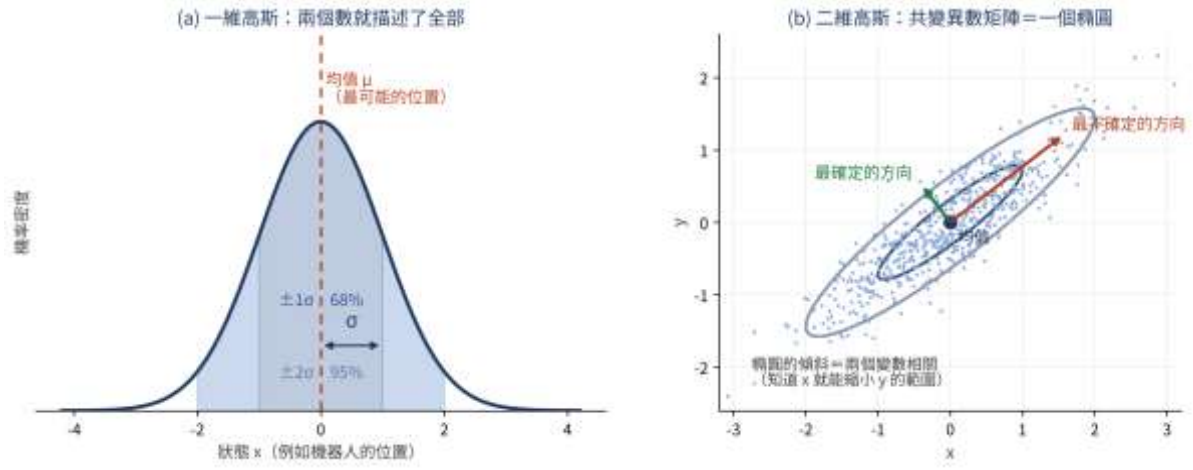
的位置），而不確定性用一個共變異數矩陣（Covariance Matrix）描述。

共變異數矩陣的幾何意義非常直觀：它對應一個不確定性橢圓。橢圓的中心是均值、大小反映不確定程度、形狀與方向反映「哪個方向比較不確定」。

- 對角線元素：各軸自己的變異數（x 方向多不確定、y 方向多不確定）。
- 非對角線元素（共變異數）：x 與 y 的相關性。若為正，表示「x 偏大時 y 也傾向偏大」，橢圓會傾斜。

舉個 SLAM 中的實例：一台機器人沿走廊直行，它對「走了多遠」（沿走廊方向）很不確定（誤差累積），但對「有沒有偏離走廊中線」（垂直方向）相當確定（牆壁一直在校正）。它的不確定性橢圓因此是一個「沿走廊方向拉長」的細長橢圓。這個橢圓會隨機器人運動與觀測不斷變形——SLAM 的每一步，本質上都在更新這個橢圓。

圖 11-1　高斯分布：從一維鐘形曲線到二維不確定性橢圓



左圖是一維的情況。整條曲線只由兩個數決定：均值 μ（最可能的位置）與標準差 σ（有多不確定）。68% 的機率落在 ±1σ 之內、95% 落在 ±2σ 之內——這兩個數字值得背起來，因為在實務上判斷「這個估計夠不夠準」時會不斷用到。

右圖是二維的情況，也是本書之後會反覆出現的那個橢圓。注意它是傾斜的——這代表 x 與 y 兩個方向的誤差互相關聯。紅色箭頭指向橢圓的長軸，是最不確定的方向；綠色箭頭指向短軸，是最確定的方向。

這個傾斜在 SLAM 中有具體的物理意義。機器人沿著走廊前進時，橫向位置可以靠兩側的牆面精確鎖定，但縱向位置卻只能靠里程計累積——於是不確定性橢圓會沿著走廊方向被拉長。第 17.5 節的「退化」、第 21 章的「遠處的點測不準」，畫出來都是這種被拉長的橢圓。

請特別記住一件事：共變異數矩陣不是一堆抽象的數字，它就是這顆橢圓。對角線元素決定橢圓在各軸向上的大小，非對角線元素決定它傾斜多少。日後看到 P、Σ 這些符號時，腦中應該直接浮現一個橢圓。

為什麼偏愛高斯

- 兩個參數搞定：均值與共變異數就完整描述了不確定性，儲存與計算都輕量。
- 運算封閉性：兩個高斯相乘仍是高斯、高斯經線性變換仍是高斯。這讓濾波器的每一步都能保持高斯形式，不會越算越複雜。這正是卡爾曼濾波如此優雅的數學根源。
- 中央極限定理：許多獨立的小誤差疊加，結果趨近高斯。感測器雜訊往往正是眾多微小因素的總和，因此高斯假設在實務上常常成立。

但要提醒：高斯假設並非萬能。當不確定性是「多峰」的（例如機器人可能在兩個長得一樣的房間之一，但不知是哪個），單一高斯就無法描述——這正是 11.7 節粒子濾波登場的原因。

11.3 感測器雜訊與運動模型

要把機器人放進機率框架，需要兩個「模型」，它們是所有濾波器的輸入。這兩個模型分別描述機器人如何「動」與如何「看」。

一、運動模型（Motion Model）

運動模型回答：「機器人執行了某個控制（如前進一公尺、左轉 30 度）後，新的位姿會是什麼？」關鍵在於——答案不是一個確定的位姿，而是一個機率分布，因為執行控制時會有雜訊（輪子打滑、地面不平）。

數學上寫成 p(x_t | x_{t-1}, u_t)：給定前一刻位姿 x_{t-1} 與控制 u_t，新位姿 x_t 的機率分布。運動模型的效果是讓不確定性擴大——每動一步，不確定性橢圓就膨脹一點，因為又累積了新的運動雜訊。這正是第 10 章 2D 位姿複合公式，加上一個雜訊項。

二、觀測模型（Observation Model）

觀測模型回答：「如果機器人真的在某個位姿，感測器應該量到什麼？」同樣地，答案是機率分布，因為感測器有雜訊。

數學上寫成 p(z_t | x_t)：給定位姿 x_t，觀測 z_t 的機率分布。例如「若機器人在此處，光達應該量到前方 2 公尺有牆」，但實際量測會在 2 公尺附近有些抖動。觀測模型的效果是讓不確定性收縮——一個明確的觀測（摸到牆角）能大幅縮小可能位置的範圍。

兩個模型的分工

這兩個模型恰好對應 11.1 節「蒙眼走路」比喻的兩個動作：運動模型讓不確定性擴大（走路時越來越模糊），觀測模型讓不確定性收縮（摸到地標時變清晰）。SLAM 的整個過程，就是這兩個模型交替作用——動一步、看一眼、動一步、看一眼。下一節的貝氏濾波，就是把這個「交替」用數學精確地表達出來。

	運動模型	觀測模型
數學形式	p(x_t | x_{t-1}, u_t)	p(z_t | x_t)
回答的問題	動了之後會到哪	在這裡會看到什麼
輸入	前一位姿 + 控制指令	當前位姿
雜訊來源	打滑、地面、輪徑誤差	感測器量測誤差
對不確定性	擴大（膨脹）	收縮（縮小）
對應比喻	蒙眼走路	摸到牆角

表 11-1　運動模型與觀測模型的對照

11.4　Bayes Filter

貝氏濾波（Bayes Filter）是本章的核心——它是卡爾曼濾波、EKF、粒子濾波等所有濾波器的共同骨架。理解了它，後面幾節都只是它的不同實作版本。

信念：對狀態的機率描述

貝氏濾波追蹤的核心物件稱為信念（Belief），記為 bel(x_t)——它是「綜合目前所有資訊後，對當前狀態的機率分布」。信念就是 11.1 節說的那個「不確定性橢圓」的數學本體。貝氏濾波的任務，就是每來一筆新資料，就更新這個信念。

兩個步驟的遞迴

貝氏濾波以兩個步驟不斷循環，這兩步驟正對應前一節的兩個模型：

11. 預測（Prediction）：機器人執行控制 u_t，用運動模型把信念往前推。這一步讓不確定性擴大——「我動了，所以更不確定自己在哪了」。數學上是把前一刻信念與運動模型做積分（對所有可能的前一位姿加總）。
12. 更新（Update / Correction）：感測器給出觀測 z_t，用觀測模型修正信念。這一步讓不確定性收縮——「我看到了地標，所以更確定自己在哪了」。數學上是把預測信念與觀測模型相乘，再正規化。

這個「預測—更新」的循環，正是第 1 章圖 1-3 SLAM 標準流程、第 7 章卡爾曼濾波「預測—更新」的同一件事，在此得到了精確的機率表述。同學應牢牢記住這個雙步驟結構：它是理解一切遞迴估測的鑰匙。

圖 11-2　貝氏濾波的遞迴循環

前一刻信念 bel(x_{t-1})

↓ ① 預測：套用運動模型 p(x_t | x_{t-1}, u_t)

→ 信念變寬（不確定性擴大）

預測信念（先驗）

↓ ② 更新：乘上觀測模型 p(z_t | x_t)，再正規化

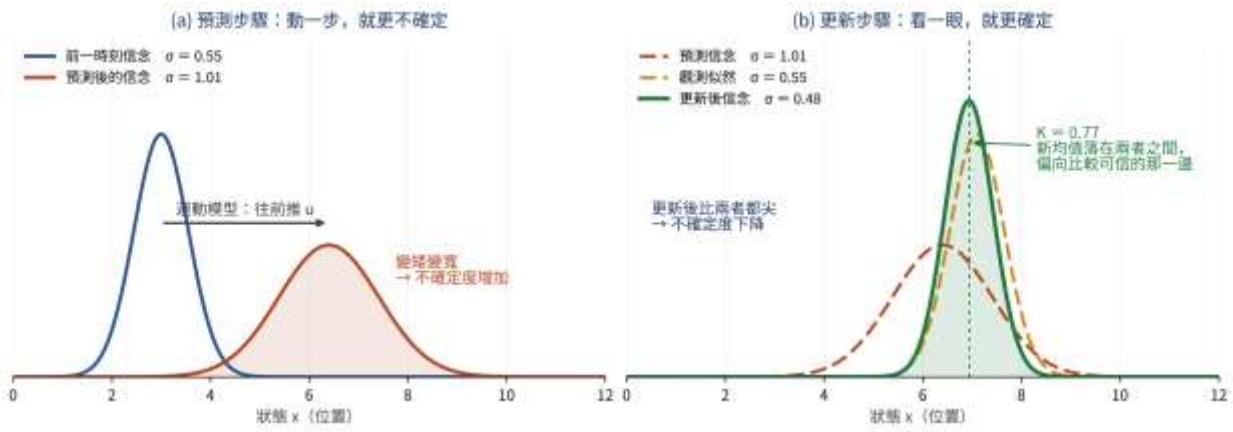
→ 信念變窄（不確定性收縮）

新信念 bel(x_t)

↓

（回到頂端，處理下一時刻）

圖 11-3　貝氏濾波的兩個步驟：預測使分布變寬，更新使分布變尖



這張圖是全章——甚至可以說是全書——最值得理解的一張。它用一維的例子把貝氏濾波的整個循環攤開來看。

左圖是預測步驟。機器人原本相當確定自己在位置 3（σ = 0.55），接著里程計說「往前走了 3.4」。於是信念的均值往前移到 6.4，但同時也變矮變寬了（σ 增為 1.01）——因為運動本身帶有雜訊，走得愈多就愈不確定。這正是第 7 章積分漂移與第 13 章里程計誤差在機

率語言中的表達：只靠遞推，不確定度只會單調上升。

右圖是更新步驟。此時感測器回報一個觀測，指出位置應該在 7.1 附近（σ = 0.55）。把預測信念與觀測似然相乘並正規化，得到的後驗信念落在兩者之間（6.94），而且比兩者都更尖（σ 降為 0.48）。

「落在兩者之間的哪裡」由卡爾曼增益決定。本例中 K = 0.77，意思是新均值往觀測那邊移動了兩者差距的 77%——因為觀測（σ = 0.55）比預測（σ = 1.01）更可信。如果觀測雜訊很大，K 就會接近 0，機器人幾乎不理會這次觀測；反之若觀測極準，K 接近 1，機器人幾乎完全採信。11.5 節會給出 K 的完整公式，但它的意義就在這張圖裡。

三個值得記住的結論

一、更新後的不確定度一定比更新前小。這是融合的根本價值——多一筆獨立觀測，就少一分不確定。

二、融合後的均值一定落在兩個來源之間，不會超出範圍。所以卡爾曼濾波永遠不會「比兩個來源都離譜」。

三、整個循環只有這兩步：預測變寬、更新變尖。第 14 章的粒子濾波、第 15 章的掃描匹配、第 23 章的視覺慣性最佳化，做的都是同一件事，只是表示法不同。

貝氏法則：更新步驟的根據

更新步驟背後的數學，就是機率論中著名的貝氏法則（Bayes' Rule）。它的精神是：把「先驗信念」（預測步驟得到的、看到觀測之前的信念）與「觀測提供的證據」結合，得到「後驗信念」（看到觀測之後、更精確的信念）。

$$\text{後驗} \propto \text{觀測似然} \times \text{先驗} \qquad bel(x_t) \propto p(z_t \mid x_t) \cdot \overline{bel}(x_t)$$

這條式子讀作：更新後的信念，正比於「觀測模型」乘上「預測信念」。它是所有濾波器更新步驟的理論根源。貝氏濾波本身是一個抽象框架，實際計算時無法直接處理任意分布

——不同的濾波器，就是對「信念用什麼形式表示、積分與相乘怎麼算」給出不同的答案。接下來三節就是三種答案。

11.5 Kalman Filter

卡爾曼濾波器（Kalman Filter,KF）是貝氏濾波在「線性系統 + 高斯雜訊」假設下的精確解。在這個假設下，信念永遠是高斯分布，於是整個濾波只需追蹤兩個量：均值（最佳估計）與共變異數（不確定性）。這讓計算極其高效，也讓卡爾曼濾波成為工程史上最成功的演算法之一——從阿波羅登月導航到今天的手機，無處不在。

卡爾曼濾波的兩步驟

它完全遵循貝氏濾波的架構，只是所有運算都在高斯的世界裡進行：

13. 預測：用運動模型把均值往前推（機器人動了，估計位置跟著移動），同時把共變異數加大（加入運動雜訊，不確定性擴大）。
14. 更新：用觀測修正均值，並把共變異數縮小。修正的力道由卡爾曼增益決定。

卡爾曼增益：全章最精妙的一個概念

卡爾曼增益（Kalman Gain,K）是整個濾波器的靈魂。它回答一個關鍵問題：預測和觀測不一致時，該相信誰、相信多少？

它的邏輯完全符合直覺：

- 若預測很不確定（共變異數大）、觀測很準（雜訊小）：增益大，多相信觀測——「我本來就沒把握，既然感測器說得這麼肯定，那就聽它的」。
- 若預測很有把握、觀測很雜：增益小，多相信預測——「我很確定自己的推算，這個雜訊很大的量測參考就好」。

換句話說，卡爾曼濾波不是死板地取平均，而是根據雙方的可信度做加權——誰更可信就更聽誰的。這個「按可信度加權」的思想，是本書反覆出現的融合精神（第 4、5、7 章）

的數學化身。第 7 章的姿態濾波、第 5 章的 GNSS/INS 組合導航，底層都是這個增益在運作。

觀念總結：卡爾曼濾波在做什麼

一句話：用「上一刻的估計 + 運動」預測現在，再用「感測器觀測」修正，

兩者按各自的可信度（共變異數）加權融合，得到比單獨任一方都更準的估計。

關鍵洞見：融合兩個帶雜訊的資訊，結果會比其中任何一個單獨都更準——

這就是為什麼「多感測器融合」永遠優於「只用最好的那一個」。

11.6 Extended Kalman Filter, EKF-SLAM

卡爾曼濾波很美，但有一個嚴格的前提：系統必須是線性的。可惜機器人的世界充滿非線性——第 10 章看過，位姿更新含 cosθ、sinθ;觀測模型「若在此處，光達會量到多遠」也是非線性的。標準卡爾曼濾波無法直接處理。EKF（擴展卡爾曼濾波）就是為此而生。

EKF 的核心技巧：局部線性化

EKF 的想法很務實：既然系統是非線性的，那就在當前估計點附近，用切線去近似曲線。數學上這叫一階泰勒展開，得到的斜率矩陣稱為雅可比矩陣（Jacobian）。線性化之後，就能套用標準卡爾曼濾波的公式。

這個近似在「非線性不太嚴重、不確定性不太大」時效果良好，是機器人領域最廣泛使用的估測方法——第 7 章提過的無人機飛控 EKF、ROS 的 robot_localization，底層都是它。但它的近似本質也帶來限制：當非線性很強或不確定性很大時，切線偏離曲線太多，估計可能失準甚至發散。

EKF-SLAM：把地圖也放進狀態

EKF 如何用來做 SLAM?關鍵一步是：把機器人位姿和所有地標的位置，通通放進同一個狀態向量，用同一個高斯分布（同一個大共變異數矩陣）一起估計。

這個設計精妙地抓住了 SLAM 的本質——第 1 章強調的「定位與建圖互相耦合」。當機器人觀測到一個地標並修正自己的位置時，共變異數矩陣中的相關性會把這個修正「傳播」到所有相關的地標與位姿。這正是閉環修正能一次攤平整張地圖誤差的數學機制：回到熟悉地點時，一個強觀測透過共變異數的相關性，同時校正了機器人與沿途所有地標。

EKF-SLAM 的歷史地位與限制

EKF-SLAM 是 SLAM 發展史上第一個完整可行的解法（1990 年代），奠定了整個領域的理論基礎，地位崇高。但它有兩個致命限制，使它逐漸退出主流：

- 計算複雜度：共變異數矩陣的大小隨地標數量平方成長。地標一多（數千個），矩陣運算就慢到無法即時——這對動輒數萬地標的大場景是硬傷。
- 線性化誤差累積：EKF 每步都做線性近似，誤差會累積，長期或大場景下可能導致估計不一致。

這兩個限制，催生了後續的兩條主流路線：一是第 14 章的粒子濾波（用另一種方式處理），二是第 12 章的圖優化（用批次最佳化取代遞迴濾波）。可以說，理解 EKF-SLAM 的限制，就理解了現代 SLAM 為何走向圖優化——這是承先啟後的關鍵一章。

11.7 Particle Filter 與 Monte Carlo Localization

卡爾曼家族都假設信念是「單一高斯」。但 11.2 節末尾提過：有些情況信念是多峰的——機器人可能在兩個一模一樣的房間之一，信念應該是「兩個高峰」。單一高斯無法描述這種情況。粒子濾波（Particle Filter）提供了截然不同的思路。

用一群樣本代表信念

粒子濾波的核心洞見是：不用數學公式描述分布，而是用一大群樣本（粒子）來「近似」它。每個粒子代表「機器人可能在這裡」的一個假設。粒子密集的地方，就是可能性高的地方。

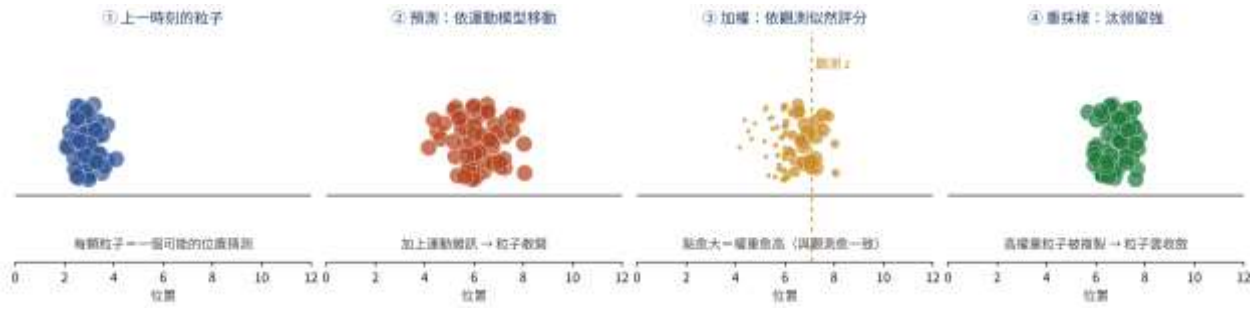
這個表示法的最大優點是：它能描述任意形狀的分布——多峰、不規則、非高斯都沒問題。這是卡爾曼家族做不到的。

粒子濾波的三步驟

它同樣遵循貝氏濾波架構，但以粒子的方式實現：

15. 預測（移動粒子）：每個粒子依運動模型移動，並各自加上隨機雜訊。於是原本聚集的粒子雲會散開——對應不確定性擴大。
16. 更新（秤重）：用觀測模型給每個粒子打分數（權重）。與觀測吻合的粒子（例如「這個位置預期看到的牆，和實際量到的相符」）得高分，不吻合的得低分。
17. 重採樣（Resampling）：依權重重新抽選粒子——高分粒子被複製多份，低分粒子被淘汰。於是粒子雲逐漸聚集到最可能的位置。這是粒子濾波的精髓：適者生存。

圖 11-4　粒子濾波的四個步驟：用一群樣本代表機率分布



粒子濾波放棄了「用公式描述分布」的做法，改用一群樣本來代表它。每一顆粒子就是一個完整的狀態猜測——在定位問題中，就是一個「機器人可能在這裡，朝這個方向」的假設。粒子密集的地方，代表機率高。

四個步驟與圖 11-3 的兩步驟完全對應。① 是上一時刻的粒子雲。② 預測：每顆粒子各自依運動模型往前移動，並加上隨機雜訊——結果是粒子散開，對應高斯版本中的「變寬」。

③ 加權：依觀測似然替每顆粒子評分，與觀測一致的粒子得到高權重（圖中點愈大）。④ 重採樣：依權重重新抽樣，高權重的粒子被複製多份、低權重的被淘汰，粒子雲因而收斂——對應高斯版本中的「變尖」。

那麼它比卡爾曼濾波好在哪？關鍵在於粒子雲可以是任何形狀。卡爾曼濾波被鎖死在「單峰高斯」裡，而真實的定位問題經常是多峰的：機器人剛開機時可能在建築物的任何地方；走在對稱的走廊中時，兩個位置的觀測看起來一模一樣。這些情境下「一個均值加一個橢圓」根本無法表達，粒子雲卻可以同時在兩個地方各留一群粒子，等到更多觀測進來再決定留哪一群。

代價是運算量。粒子數必須隨狀態維度指數成長，所以粒子濾波在二維定位（三個自由度）上很實用，到了高維度就撐不住。這正是第 14 章 Gmapping 必須採用 Rao-Blackwell 化技巧的原因——只對位姿用粒子，地圖的部分改用解析方法處理。

蒙地卡羅定位（MCL）與 AMCL

把粒子濾波用於「在已知地圖中定位」，就是蒙地卡羅定位（MCL,Monte Carlo Localization），ROS 中的實作稱為 AMCL（第 3 章提過）。它的運作極為直觀：一開始不知道機器人在哪，就在地圖各處撒滿粒子；機器人一邊移動一邊掃描，與觀測不符的粒子被淘汰，粒子雲逐漸收斂到真實位置。

MCL 天生就能解決第 3 章的「綁架機器人問題」：即使機器人被搬走，只要在地圖各處保留一些粒子，它們就能在新位置重新「勝出」，讓機器人找回自己。這是單一高斯的卡爾曼濾波難以做到的——這也是掃地機器人被抱到別的房間後能重新定位的數學原因。

粒子濾波與 SLAM：預告 Gmapping

把粒子濾波用於 SLAM（而非只是定位），需要一個巧妙的技巧，稱為 Rao-Blackwellized 粒子濾波（RBPF）——每個粒子不只代表一個位姿，還各自攜帶一整張地圖。這正是第 14 章 Gmapping 的核心，也是本書第四篇 2D SLAM 的主角。本章先把粒子濾波

的原理講清楚，第 14 章就能專注於它如何建圖。

特性	卡爾曼 / EKF 家族	粒子濾波
信念表示	單一高斯（均值 + 共變異數）	一群粒子樣本
可處理分布	限高斯（單峰）	任意形狀（可多峰）
非線性	EKF 靠線性化近似	天然支援，不需線性化
綁架問題	難處理	天生可解
計算成本	低（維度低時）	高（需大量粒子）
代表應用	GNSS/INS、EKF-SLAM	AMCL、Gmapping

表 11-2　卡爾曼家族與粒子濾波的比較

至此，同學已經掌握了 SLAM 的核心估測工具：用機率描述不確定性，用貝氏濾波的「預測—更新」處理它，並認識了卡爾曼、EKF、粒子濾波三種具體實作。這些是遞迴濾波的路線——每來一筆資料就更新一次。但現代 SLAM 還有另一條更強大的路線：不急著逐筆處理，而是累積一批資料後一次性最佳化。下一章「最佳化與圖論 SLAM」，將介紹這條主宰當代 SLAM 的路線——Pose Graph、Bundle Adjustment 與 g2o、Ceres、GTSAM。

公式或流程

一、一維高斯分布

一維高斯分布的機率密度函數：

$$p(x) = (1 / \sqrt{(2\pi \sigma^2)}) \cdot exp(- (x - \mu)^2 / (2\sigma^2))$$

其中 μ 為均值（分布中心、最佳估計）、σ^2 為變異數（不確定性大小）。約 68% 的機率落在 μ±σ、95% 落在 μ±2σ、99.7% 落在 μ±3σ——這組「68-95-99.7 法則」是判讀不確定性最實用的直覺。

二、貝氏法則

狀態估測的理論核心：

$$p(x \mid z) = p(z \mid x) \cdot p(x) \;/\; p(z)$$

讀作：後驗 = (似然 × 先驗) / 證據。在濾波中，p(x) 是預測步驟得到的先驗信念、p(z|x) 是觀測模型、p(x|z) 是更新後的後驗信念。分母 p(z) 只是正規化常數（確保機率總和為 1），因此常寫成正比形式：

$$p(x \mid z) \propto p(z \mid x) \cdot p(x)$$

三、貝氏濾波兩步驟

預測步驟（套用運動模型，對前一狀態積分）：

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t) \cdot bel(x_{t-1})\, dx_{t-1}$$

更新步驟（乘上觀測模型並正規化）：

$$bel(x_t) = \eta \cdot p(z_t \mid x_t) \cdot \overline{bel}(x_t)$$

其中 $\overline{bel}$ 是預測信念（先驗）、bel 是更新後信念（後驗）、η 是正規化常數。卡爾曼濾波、EKF、粒子濾波都是這兩條式子在不同表示法下的具體實現。

四、卡爾曼濾波核心方程（概念版）

預測：

$$\text{均值：} \bar{x} = F \cdot \hat{x} \qquad \text{共變異數：} \bar{P} = F \cdot P \cdot F^{T} + Q$$

更新：

$$\text{卡爾曼增益：} K = \bar{P} \cdot H^{T} \cdot \left(H \cdot \bar{P} \cdot H^{T} + R \right)^{-1}$$

$$\text{均值：} \hat{x} = \bar{x} + K \cdot \left(z - H \cdot \bar{x} \right) \qquad \text{共變異數：} P = \left(I - K \cdot H \right) \cdot \bar{P}$$

其中 F 是運動模型、H 是觀測模型、Q 是運動雜訊、R 是觀測雜訊。細看增益 K 的式子：當觀測雜訊 R 很小（觀測很準），K 變大，更新時就更相信觀測（z - H·x̄ 這個「觀測與預測的差」被乘上大的 K）；當 R 很大，K 變小，更相信預測。這條式子把 11.5 節「按可信度

加權」的直覺精確地數學化了。

五、三種濾波器選擇流程

圖 11-5 濾波器選擇決策流程

問題一：系統是線性的嗎？

├─ 是 → 卡爾曼濾波（KF）：最佳、最快

└─ 否 → 進入下一問

↓

問題二：非線性溫和、信念單峰、有好的初始估計嗎？

├─ 是 → 擴展卡爾曼濾波（EKF）：線性化近似，高效

└─ 否（強非線性 / 多峰 / 綁架問題）→ 進入下一問

↓

問題三：狀態維度低、可負擔大量粒子嗎？

├─ 是 → 粒子濾波（PF / MCL）：能處理任意分布

└─ 否（高維、大場景）→ 考慮第 12 章的圖優化

11.8 狀態估測的當代發展：濾波沒有退場，而是換了位置

本章介紹的卡爾曼家族與粒子濾波，在教科書上常被描述為「經典方法」，彷彿已被第 12 章的圖優化取代。這個印象只對了一半，值得同學仔細分辨。

一、後端確實被最佳化接管了

在 SLAM 的後端——也就是維護整條軌跡與整張地圖的那一層——最佳化確實已經全面勝出。原因在 11.6 節提過：EKF-SLAM 的共變異數矩陣隨地標數量平方成長，幾千個地標就

撐不住；而且線性化只做一次，早期的錯誤無法回頭修正。

第 12 章的圖優化保留所有歷史位姿與觀測，可以反覆重新線性化，還能在閉環時一次修正整條軌跡。再加上 iSAM2 這類增量式求解器只更新受影響的部分，使得最佳化在速度上也不再吃虧。今天的 ORB-SLAM3、Cartographer、LIO-SAM 無一例外採用這條路線。

二、但前端與高頻迴路仍然是濾波的天下

然而把視角拉近到「每秒數百次的即時估測」，濾波仍然不可取代。無人機飛控的姿態估測、四足機器人的機身狀態估測、第 7 章提到的 IMU 高頻遞推，都還是 EKF 在跑——因為最佳化需要累積一批資料才能求解，而控制迴路等不了。

這帶出一個實務上的分工，同學應該建立起來：高頻、低維度、要求即時 → 濾波；低頻、高維度、要求全域一致 → 最佳化。一個完整的機器人系統兩者都有，而且是分層並存，不是二選一。第 18 章的 LIO-SAM、第 23 章的 ORB-SLAM3 都是這種分層架構。

三、不變式 EKF：讓濾波重新變得可靠

濾波本身也在進步。11.6 節提到 EKF 的根本弱點是線性化誤差——當估計偏離真值較遠時，雅可比矩陣算在錯的地方，濾波可能發散。近年的不變式擴展卡爾曼濾波（InEKF）利用第 10.8 節提到的李群結構，把狀態定義在矩陣李群上；其誤差動態在特定條件下與估計值無關，因而具有更好的收斂性質。

這條路線在足式機器人上特別成功，已成為許多四足與人形機器人機身狀態估測的標準做法。對同學而言，它是一個很好的例證：第 10 章看似抽象的李群，在這裡直接換成了更穩定的濾波器。

四、神經網路補償：讓濾波處理它建模不了的東西

最新的一條路線，是把學習模型掛在濾波器旁邊，專門處理模型假設失效的情況。2026 年一項針對足式機器人的研究就是典型例子：它在不變式 EKF 之上加了一個以注意力機制為核心的神經補償器。

其設計邏輯值得細看。足式機器人的狀態估測仰賴「觸地的腳不會滑動」這個假設，但在碎石、低摩擦面上這個假設會崩壞。該研究把腳底打滑程度當成明確的條件變數，以 GRU 自動編碼器分別編碼濾波器的狀態歷史與打滑程度，再用交叉注意力產生依打滑情境調整的補償量，最後在濾波器的後驗狀態上加一個九維的殘差修正。

效果相當顯著：在碎石場地上位置誤差較最強基準降低約 93%，在低摩擦的鐵氟龍板上降低約 49%，約一百公尺的室外草地測試降低約 55% 至 73%，而整體仍可在 580 赫茲下即時運行。

請注意它的結構——與第 7.8 節的學習式 IMU、第 9.8 節的深度估測模型完全一致：保留幾何與機率的骨架，讓神經網路負責補償那些「物理模型寫不出來」的部分。這已經是目前機器人領域最穩健的 AI 整合模式，第 32 章會從更高的層次再討論一次。

> **一句話總結這四件事**
>
> 濾波沒有被取代，它只是從「負責整張地圖」退回到「負責即時的那一小段」——而在那個位置上，它仍然無可取代。
>
> 同學學本章不是為了考古：你在第 26 章寫無人機姿態估測、在第 29 章調 AMCL 參數、在第 16 章理解掃地機器人為什麼會「迷路後突然想起自己在哪」，用的都是本章的東西。

案例

案例 11-1　阿波羅登月：卡爾曼濾波的成名之戰

卡爾曼濾波器由 Rudolf Kálmán 於 1960 年提出，而它第一個震撼世界的應用，就是阿波羅登月計畫。太空船在地月之間航行，導航系統必須融合慣性量測與地面追蹤等多種帶雜訊的資料，即時估出太空船的位置與速度——這正是卡爾曼濾波「按可信度融合多源資訊」的完美舞台。

當時太空船的運算能力極其有限（遠不如今天一支計算機），但卡爾曼濾波只需追蹤均值與共變異數兩個量、且是遞迴計算（不必儲存全部歷史資料），恰好能在如此拮据的硬體上即時運行。可以說，沒有卡爾曼濾波，阿波羅的導航幾乎無法實現。

這個案例呼應了第 10 章的阿波羅萬向鎖——同一個登月任務，同時推動了旋轉表示法與狀態估測兩大數學的發展。它也給同學一個啟示：本章這些看似抽象的機率工具，是人類踏上月球的實際基石。六十多年後的今天，你手機裡的定位、掃地機器人的導航，用的仍是同一套數學。

案例 11-2　掃地機器人的「粒子雲」：AMCL 的視覺化

許多支援地圖顯示的掃地機器人 App，若進入除錯或進階模式，能看到一個生動的畫面：機器人開機時，地圖上散布著大量代表「可能位置」的小點（粒子）；隨著機器人移動幾步、掃描幾次，這些小點迅速從「散布全圖」收斂成「緊密的一小團」——機器人「想通了」自己在哪。

這正是 11.7 節 AMCL 的即時視覺化。散開的粒子代表「我還不確定在哪」，收斂的粒子團代表「我很確定了」。如果把機器人抱到別的房間，會看到粒子團先是「困惑」（與觀測不符），然後在正確的新位置重新聚集——綁架問題的解決過程活生生地展現在眼前。

這個案例的教學價值在於：它把本章最抽象的「機率分布」變成看得見的粒子雲。大學課堂上，教師常讓同學在 RViz 中觀察 AMCL 的粒子演化，配合本章理論，抽象的「信念收縮」瞬間變得具體。這也是本書強調「先理解原理、再動手觀察」的又一個例子。

案例 11-3　自駕車的感測器融合：增益在真實道路上的權衡

一台自駕車同時擁有 GPS、IMU、輪速計、光達、相機多種感測器，它們對「車現在在哪」各有各的說法，而且各有各的可靠時機。卡爾曼濾波家族（通常是 EKF 或其變體）正是融合這些資訊的主力。

本章的卡爾曼增益概念，在此有極為真實的體現：當車輛開進隧道，GPS 失鎖（觀測雜訊 R 飆升），濾波器自動調低對 GPS 的信任、改倚重 IMU 與輪速計的推算（第 5 章的組合導航）；出隧道後 GPS 恢復，濾波器又逐漸把信任轉回來。這個「依情境動態調整信任」的過程，不需要工程師寫死規則，而是共變異數與增益的數學自然導出的結果。

這正是機率方法相對「寫死規則」的根本優勢：它讓系統依據每個資訊源當下的可信度自動權衡，優雅地適應各種情境。這個思想貫穿全書——從第 4 章的室內定位融合、第 5 章的 GNSS/INS、第 7 章的姿態濾波，到本章的統一數學表述，同學應該能感受到：整個第二篇與第三篇，其實都在講同一件事的不同層面。

實作活動

實作 11-1　一維卡爾曼濾波與粒子濾波動手做

目的：用最簡單的一維例子，親手實作卡爾曼濾波與粒子濾波，親眼看見「不確定性擴大與收縮」以及「粒子雲收斂」。本實作以 Python 為主，是理解本章所有理論的關鍵一步。

第一部分：一維卡爾曼濾波（全體）

5. 設定情境：一台機器人在一維軌道上，真實位置未知。它有一個會漂移的「速度感測器」（運動模型，雜訊大）與一個偶爾回報的「位置感測器」（觀測模型，雜訊中等）。用程式產生一段帶雜訊的模擬資料。
6. 實作卡爾曼濾波的預測步驟：每步用速度更新位置均值，並把變異數加大。畫出「估計位置」與「不確定範圍（均值±標準差）」隨時間的變化，觀察只有預測時不確定範圍如何持續擴大。
7. 加入更新步驟：每當位置感測器回報，就用觀測修正均值、縮小變異數。觀察不確定範圍在每次觀測時如何「收縮」。
8. 把「純預測（航位推算）」與「完整卡爾曼濾波」的估計軌跡畫在一起，與真值比較，量化卡爾曼濾波減少了多少誤差。

9. 實驗卡爾曼增益：把觀測雜訊 R 分別設得很大與很小，觀察增益如何變化、估計軌跡如何更偏向預測或更偏向觀測。

第二部分：一維粒子濾波（全體或大學部）

10. 在一維軌道上撒 500 個粒子（均勻分布），代表「一開始不知道機器人在哪」。
11. 實作三步驟：每步讓所有粒子依運動模型移動（加雜訊）、依觀測給每個粒子打權重、再依權重重採樣。
12. 每幾步畫一次粒子分布的直方圖，觀察粒子雲如何從「散布全軌道」收斂到「真實位置附近」。
13. 綁架測試：在中途突然把真實位置換到別處，觀察粒子雲如何先「困惑」、再重新聚集到新位置。

第三部分：多峰情境（大學部）

14. 設計一個「對稱走廊」情境：兩個地點的觀測完全相同（對稱環境）。用粒子濾波處理，觀察粒子雲如何維持「兩個高峰」——親身驗證 11.7 節「粒子濾波能處理多峰、卡爾曼不能」的論述。
15. 思考：若用卡爾曼濾波處理這個對稱情境，會發生什麼？（提示：單一高斯會把均值放在兩峰中間——一個機器人根本不可能在的位置。）

討論問題

8. 在第一部分，完整卡爾曼濾波比純航位推算減少了多少誤差？這個改善來自哪裡？
9. 當你把觀測雜訊 R 調得很小，估計軌跡發生什麼變化？這符合 11.5 節對卡爾曼增益的描述嗎？
10. 粒子濾波的綁架測試中，粒子雲花了多久重新聚集？若粒子數只有 50 個（而非 500），結果會如何？這說明粒子數的什麼取捨？
11. （大學部）在對稱走廊情境，粒子濾波維持了兩個高峰，而卡爾曼濾波會失敗。這對「選擇哪種濾波器」有什麼實務啟示？

延伸挑戰（大學部）

把一維卡爾曼濾波擴展到二維（x、y），追蹤共變異數矩陣，並用 matplotlib 畫出每一步的「不確定性橢圓」，觀察機器人沿直線運動時橢圓如何沿運動方向拉長、觀測後如何收縮。這個二維視覺化直接對應 11.2 節，也為第 14 章 Gmapping 與第 29 章 AMCL 實作打下直覺基礎。

延伸閱讀

8. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005.（機率機器人學聖經，本章全部主題的權威來源，第 2 至 8 章尤其相關）
9. Cyrill Stachniss, SLAM 課程之 Bayes Filter、Kalman Filter、Particle Filter 系列影片（YouTube）.（以動畫直觀講解本章三種濾波器，自學首選）
10. R. Faragher, "Understanding the Basis of the Kalman Filter via a Simple and Intuitive Derivation," IEEE Signal Processing Magazine, 2012.（卡爾曼濾波最友善的直觀推導）
11. R. Labbe, "Kalman and Bayesian Filters in Python"（GitHub 免費電子書）.（附完整可執行程式，實作本章的最佳伴讀）
12. D. Fox et al., "Monte Carlo Localization for Mobile Robots," ICRA, 1999.（蒙地卡羅定位 MCL 的原始論文）
13. CMU 16-833 Robot Localization and Mapping 課程（第 2 章介紹）之濾波單元.（EKF-SLAM 與粒子濾波的研究所級教材）
14. 高翔、張濤，《視覺 SLAM 十四講》第 6 講（非線性最佳化）前段.（銜接本章濾波與第 12 章圖優化的橋樑）

從濾波到最佳化：本章的位置回顧

本章把 SLAM 的問題寫成了一個機率問題，並給了三種解法。但這三種解法有一個共同的性

格：它們都是遞迴的——只保留「當前」的信念，每來一筆新資料就更新它，然後把舊資料丟掉。

這樣做的好處是記憶體固定、運算量穩定，非常適合即時系統。但代價也很明確：一旦某一步的估計錯了，錯誤就被壓進了當前的信念裡，再也無法回頭修正。機器人繞了一大圈回到起點，發現地圖對不上時，濾波器只能從那一刻起慢慢修正，卻無法回頭把中間那一整段軌跡一起扳正。

第 12 章要介紹的是完全相反的思路：不丟掉任何東西。把每一個時刻的位姿都當成一個待求的變數，把每一筆觀測都當成一條約束，全部放進一個巨大的最佳化問題中一次求解。閉環偵測到「我回到起點了」的那一刻，這條新約束會把整條軌跡一起拉回正確的位置——這是濾波永遠做不到的事。

有趣的是，兩者的數學根源相同：最佳化求的最大後驗估計，與濾波追求的後驗分布峰值，在高斯假設下是同一件事。第 12 章會說明這個等價關係。所以本章學到的一切都不會白費——它只是換了一種計算方式。

小結

1. SLAM 必須以機率處理不確定性：每個感測器給的都是帶雜訊的猜測，成熟的做法是量化不確定性再做最佳權衡。關鍵觀念是——估計的是「機率分布」（最佳猜測 + 可信度），而非單一數值。
2. 高斯分布以均值（最佳估計）與變異數（不確定性）兩參數完整描述；多維時共變異數矩陣對應「不確定性橢圓」。高斯因兩參數搞定、運算封閉、中央極限定理而成為 SLAM 首選，但無法描述多峰分布。
3. 運動模型 p(x_t|x_{t-1},u_t) 描述「動了會到哪」使不確定性擴大；觀測模型 p(z_t|x_t) 描述「在這裡會看到什麼」使不確定性收縮。兩者是所有濾波器的輸入。

4. 貝氏濾波是所有濾波器的共同骨架，以「預測（套運動模型、信念變寬）—更新（套觀測模型、信念變窄）」兩步驟遞迴，更新步驟的根據是貝氏法則：後驗 ∝ 似然 × 先驗。

5. 卡爾曼濾波是線性高斯下的最佳解，只需追蹤均值與共變異數；卡爾曼增益依雙方可信度權衡預測與觀測——這是「多感測器融合優於單一感測器」的數學核心。

6. EKF 以雅可比矩陣局部線性化處理非線性，是機器人最廣用的估測法；EKF-SLAM 把位姿與地標放進同一狀態聯合估計，體現定位與建圖的耦合，但共變異數隨地標平方成長且線性化誤差累積，催生了圖優化路線。

7. 粒子濾波以一群樣本近似任意分布，經預測—秤重—重採樣三步驟，能處理多峰與非線性、天生可解綁架問題；蒙地卡羅定位（AMCL）是其定位應用，Rao-Blackwellized 版本（Gmapping）則是其 SLAM 應用。

本章的濾波器都是「遞迴」的——每來一筆資料就更新一次，處理完就丟掉。這高效，但也意味著過去的資訊一旦被壓縮進當前信念，就無法再回頭修正。下一章「最佳化與圖論 SLAM」介紹另一條路線：把一批位姿與觀測全部保留，建成一張「位姿圖」，一次性最佳化求出最一致的解。這條路線精度更高、能更好地處理閉環，正是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 12 章

最佳化與圖論 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第 11 章把 SLAM 寫成了機率問題，並給出三種濾波解法。但濾波有一個共同的性格：它只保留「當前」的信念，每更新一次就把舊資料丟掉。這在即時性上很划算，在正確性上卻付出代價——錯誤一旦被壓進信念裡，就再也無法回頭修正。

本章要介紹的是相反的思路：什麼都不丟。把每一個時刻的位姿都當成一個待求的變數，把每一筆觀測都當成一條約束，全部放進同一個最佳化問題中一次求解。當機器人繞一圈回到起點、發現地圖對不上時，這條新的閉環約束能把整條軌跡一起扳正——這是濾波永遠做不到的事。

這就是現代 SLAM 的主流架構。第 14、15 章的 2D SLAM、第 18 章的光達慣性里程計、第 22、23 章的 ORB-SLAM 系列，後端全都是本章的圖優化。可以說，第三篇三章當中，本章是最直接連到實作的一章。

圖 12-0　第 12 章與全書的連結地圖

【承接前章】

第 10 章　位姿 T　→　本章成為圖的節點

第 11 章　機率與資訊　→　本章成為邊的資訊矩陣 Ω

第 11 章的濾波「丟掉歷史」　→　本章「保留全部歷史」

【本章負責】

前端 / 後端分工　→　位姿圖　→　最小平方　→　Bundle Adjustment

稀疏性：為什麼上萬個節點也解得動

g2o、Ceres、GTSAM：真正會用到的工具

【往下延伸】

→ 第 15 章　Cartographer 的分支定界閉環 + 位姿圖後端

→ 第 17 章　ICP 配準結果 = 位姿圖的一條邊

→ 第 18 章　LIO-SAM：光達、IMU、GNSS 的因子圖

→ 第 22、23 章　ORB-SLAM 的局部 BA 與全域 BA

→ 第 7.5 節　IMU 預積分的產物，就是本章的一條邊

學習目標

上一章的濾波器是「遞迴」的——每來一筆資料就更新一次，處理完就丟掉。本章介紹另一條路線：把一批位姿與觀測全部保留，建成一張「圖」，一次性最佳化求出最一致的解。這條圖優化路線精度更高、更能處理閉環，是 ORB-SLAM、Cartographer 等當代主流系統的數學基礎，也是第三篇的收尾。學完本章之後，同學應能夠達成下列目標：

14. 說明 SLAM 前端與後端的分工，理解圖優化屬於後端。
15. 說明位姿圖（Pose Graph）的節點與邊代表什麼，並看懂它如何描述一個 SLAM 問題。

16. 解釋為什麼 SLAM 可以化為一個最小平方問題。

17. 說明 Bundle Adjustment 的概念與它在視覺 SLAM 中的角色。

18. 認識 g2o、Ceres、GTSAM 三大最佳化函式庫的定位與用途。

19. 說明為什麼現代 SLAM 多採圖優化，而非上一章的濾波器。

核心概念

中文名詞	英文名詞	概念說明
前端	Front-End	把原始感測資料轉成幾何約束（特徵、匹配、里程計）。
後端	Back-End	在所有約束下求出最一致的位姿與地圖。
位姿圖	Pose Graph	以節點表示位姿、邊表示約束的圖結構。
節點	Node / Vertex	待估計的量，如機器人位姿或地標位置。
邊	Edge / Constraint	兩節點之間的相對量測約束。
最小平方	Least Squares	尋找使所有誤差平方和最小的解的方法。
殘差	Residual	量測值與預測值之間的差異。
光束法平差	Bundle Adjustment, BA	同時最佳化相機位姿與三維點的視覺最佳化。
重投影誤差	Reprojection Error	三維點投影到影像與實際觀測像素的差距。
資訊矩陣	Information Matrix	共變異數的反矩陣，代表約束的可信度權重。
稀疏性	Sparsity	每個約束只牽涉少數變數，使大問題可高效求解。
圖優化函式庫	g2o / Ceres / GTSAM	求解圖優化問題的開源工具。

12.1 前端與後端的分工

現代 SLAM 系統普遍採用「前端 + 後端」的兩段式架構（第 2 章各校課程的共同骨架、第 1 章圖 1-3 的核心）。理解這個分工，是讀懂本章的第一步。

前端：把感測資料變成幾何約束

前端（Front-End）直接面對感測器，它的工作是把原始資料轉成「幾何約束」——也就是「A 位置到 B 位置之間，相對移動了多少」或「在 A 位置看到了某個地標」這類幾何關係。

- 對光達 SLAM：前端做掃描匹配（第 13 章），算出「這一幀相對上一幀移動了多少」。
- 對視覺 SLAM：前端做特徵提取與匹配（第 9、20 章），算出相機運動與看到的地標。
- 前端還負責閉環偵測：辨識「這個地方我來過」，產生一條關鍵的閉環約束。

後端：在所有約束下求最一致解

後端（Back-End）不直接碰感測器，它接收前端產生的一堆幾何約束，任務是找出一組位姿與地圖，使它們「盡可能同時滿足所有約束」。

問題來了：這些約束往往互相矛盾。里程計說「我從 A 走到 B 移動了 10 公尺」，但閉環偵測說「B 其實就在 A 旁邊」——兩個約束打架。後端的工作，就是在這些矛盾的約束之間找一個「最佳妥協」，讓總體誤差最小。這正是本章的主題。

兩種後端路線

如何求這個「最佳妥協」，歷史上有兩條路線：

- 濾波路線（第 11 章）：遞迴地逐筆處理，以 EKF、粒子濾波為代表。優點是即時、記憶體固定；缺點是過去資訊被壓縮後無法回頭修正。

- 最佳化路線（本章）：保留一批約束，一次性求解整個問題，以圖優化為代表。優點是精度高、能反覆修正、能漂亮地處理閉環；代價是計算量較大。

過去十餘年，最佳化路線因為精度與閉環處理的優勢，已成為主流。本章就是要說清楚這條路線的數學與工具。

12.2　Pose Graph 的概念

圖優化 SLAM 的核心資料結構，是一張「位姿圖」（Pose Graph）。它把抽象的 SLAM 問題，變成一張直觀的「圖」——就是資料結構課學過的、由節點與邊構成的圖。

節點與邊

- 節點（Node / Vertex）：代表「待估計的量」。在最單純的位姿圖中，每個節點是機器人在某個時刻的位姿（第 10 章的 x、y、θ 或 6-DoF）。在含地標的版本中，地標位置也是節點。這些就是我們想求出來的未知數。
- 邊（Edge / Constraint）：代表兩個節點之間的「相對量測約束」。它記錄「根據某個感測器，這兩個節點之間應該是什麼關係」。

兩種關鍵的邊

位姿圖中有兩種來源不同、但同等重要的邊：

- 里程計邊（相鄰邊）：連接時間上相鄰的兩個位姿，來自里程計或掃描匹配——「我從第 t 幀走到第 t+1 幀，移動了這麼多」。這些邊像一條鏈子，把位姿串起來。
- 閉環邊（迴路邊）：連接時間上不相鄰、但空間上重逢的兩個位姿，來自閉環偵測——「我現在所在的位置，和很久以前的第 5 幀是同一個地方」。這種邊在圖上形成「迴路」，是消除累積漂移的關鍵。

為什麼閉環邊這麼重要

想像機器人繞行大樓一圈回到起點。里程計邊像一條鏈子，一路累積誤差，走完一圈後終點與起點之間出現明顯落差（第 1 章的漂移）。這時閉環偵測產生一條邊，宣告「終點就

是起點」——這條邊與整條里程計鏈子產生矛盾。

後端最佳化的魔法，就是把這個矛盾「均攤」到整條軌跡上：它微調沿途每一個位姿，讓終點回到起點，同時盡量不破壞里程計約束。結果是——整條軌跡被「拉正」，累積的漂移一次性被消除。這就是為什麼閉環對 SLAM 如此關鍵，也是圖優化相對濾波器的一大優勢：它能回頭修正所有歷史位姿。

圖 12-1　位姿圖的結構

節點：①—②—③—④—⑤—⑥—⑦—⑧ （每個是一個位姿）

└———里程計邊（相鄰，像鏈子）———┘

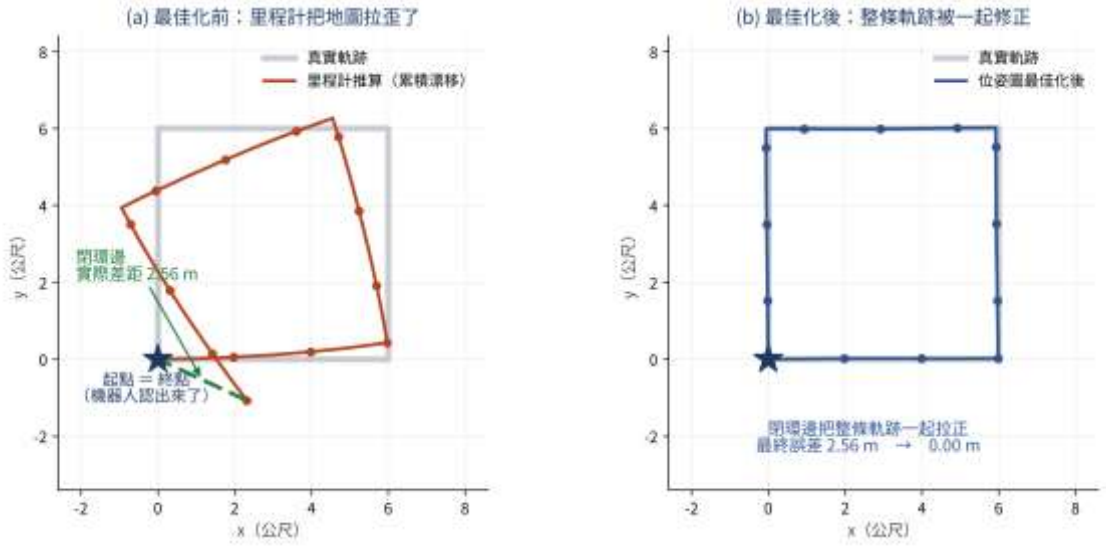
① ←————————閉環邊————————→ ⑧

（偵測到第⑧幀回到了第①幀的位置）

最佳化前：里程計鏈累積漂移，⑧ 與 ① 對不上

最佳化後：閉環邊把矛盾均攤到 ①~⑧，整條軌跡被拉正

圖 12-2　閉環的威力：一條邊如何把整張地圖拉正



這張圖是真的跑出來的，不是示意。模擬一台機器人沿著 6 公尺見方的正方形走一圈，里程計帶有每步約 0.013 弧度的系統性角度偏差——這是真實輪式機器人非常典型的誤差來源，例如左右輪胎直徑略有差異。

左圖是只靠里程計推算的結果。注意它的形狀：每一段都還算直，轉角也都接近 90 度，看起來完全合理——但整體已經轉歪了。走完一圈回到起點時，推算位置與真實起點差了 2.56 公尺。這正是第 7 章積分漂移與第 11 章「預測步驟只會讓不確定度上升」在二維平面上的樣子。

關鍵的一步發生在機器人認出「我回到起點了」的那一刻。這個認出的動作由前端完成——在第 14、15 章是掃描匹配，在第 20、22 章是視覺詞袋。前端給出的結論寫成一條閉環邊：節點 N 與節點 0 之間的相對位姿應該是零。

右圖是加入這條邊之後最佳化的結果。整條軌跡被一起拉正，最終誤差從 2.56 公尺降到 0.00 公尺，全程平均誤差也從 1.18 公尺降到 0.04 公尺。請注意「一起」兩個字——被修正的不只是最後一個節點，而是中間每一個節點都分擔了修正量。這就是圖優化相對於濾波最根本的優勢：它能回頭改寫歷史。

為什麼濾波做不到這件事

濾波器在第 k 步就把第 k 步以前的資訊壓縮成一個均值與一個共變異數，中間節點的位置已經不存在於狀態中，自然無從修正。

圖優化保留了全部 N 個節點作為變數，閉環邊提供的新資訊可以透過一連串里程計邊「傳播」回去，讓每個節點都移動一點點。

代價是變數變多、要存全部歷史。這就是 12.7 節所說的取捨，也是為什麼實務上常把兩者分層並用：前端用濾波維持高頻，後端用圖優化維持全域一致。

12.3 節點、邊、限制條件

本節更仔細地看邊所代表的「限制條件」，以及如何量化「約束被滿足的程度」——這是把位姿圖變成可求解問題的關鍵。

殘差：量測與預測的差

每一條邊都帶有一個量測值（感測器說兩節點之間應該是什麼關係）。而根據當前對兩節點位姿的估計，我們也能算出一個預測值（按目前估計，兩節點之間實際是什麼關係）。兩者的差，稱為殘差（Residual）：

$$\textit{殘差} = \textit{量測值} - \textit{預測值}$$

殘差為零，表示這條邊的約束被完美滿足；殘差越大，表示當前的位姿估計越違背這條約束。後端的目標，就是調整所有節點，讓所有邊的殘差盡量小。

不是所有約束都一樣可信：資訊矩陣

這裡有一個關鍵細節：不同的邊可信度不同。一條來自高精度光達掃描匹配的邊，遠比一條來自打滑里程計的邊可信。最佳化時，應該更「尊重」可信的邊。

這個可信度用資訊矩陣（Information Matrix）描述——它正是第 11 章共變異數矩陣的反矩陣。共變異數大（不確定）對應資訊矩陣小（權重低）；共變異數小（確定）對應資

訊矩陣大（權重高）。於是可信的約束在最佳化中「說話比較大聲」。這讓圖優化與第 11 章的機率框架完美接軌：兩者其實是同一個機率問題的不同解法。

限制條件的一致性

位姿圖的美妙之處在於：它把各種來源的約束——里程計、掃描匹配、閉環、甚至第 4 章的 UWB 測距、第 5 章的 GNSS、第 7 章的 IMU 預積分——通通統一成「節點之間的邊」。無論約束來自哪個感測器，在圖裡都是一條帶殘差與資訊矩陣的邊。這種統一性，正是圖優化能優雅融合多感測器的原因，也呼應了前面各章反覆出現的「緊耦合」思想：把所有原始約束放進同一個最佳化問題一起求解。

12.4 Least Squares 最小平方

現在把前面的概念收攏成一個明確的數學問題。既然要讓所有邊的殘差都盡量小，自然的做法是：找一組節點值，使「所有殘差的平方和」最小。這就是最小平方（Least Squares）。

為什麼是「平方」和

為什麼不是殘差的絕對值和，而是平方和？有兩個理由：

- 數學理由：平方函數處處可微、形式漂亮，便於用微積分求極小值（令導數為零）。絕對值在零點不可微，處理起來麻煩。
- 機率理由：當雜訊是高斯分布時（第 11 章），「最小化殘差平方和」在數學上恰好等價於「最大化後驗機率」（找最可能的解）。換句話說，最小平方不是隨便選的，它是高斯假設下的最佳估計。這再次顯示圖優化與第 11 章機率框架是同一件事。

加上資訊矩陣作為權重後，更精確的說法是「最小化加權殘差平方和」——可信的邊權重大，對總和的影響也大。

如何求解：反覆逼近

這個最小平方問題通常是非線性的（因為殘差含 cosθ、sinθ 等，第 10 章），無法一步解出。實務上用迭代法反覆逼近：

18. 從一個初始猜測開始（通常用里程計串起來的粗略軌跡）。

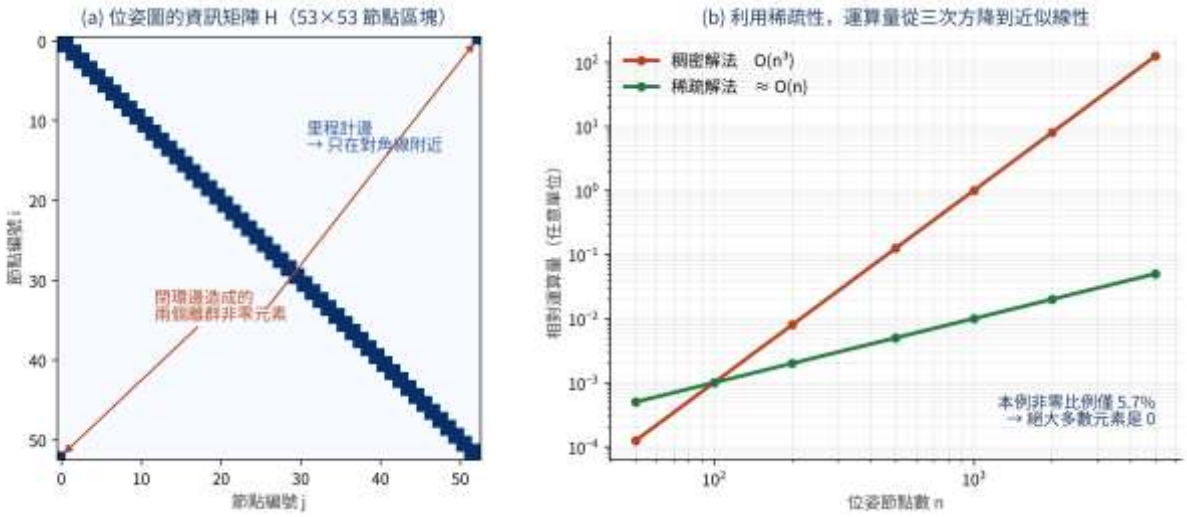
19. 在當前估計附近把問題線性化（求雅可比矩陣，第 11 章 EKF 用過同樣的技巧）。

20. 解一個線性方程組，得到一個「往誤差更小的方向修正」的步伐。

21. 更新估計，回到步驟 2，反覆進行直到殘差不再明顯下降（收斂）。

常用的迭代方法有高斯—牛頓法（Gauss-Newton）與列文伯格—馬夸特法（Levenberg-Marquardt）；後者更穩健，是許多 SLAM 系統的預設選擇。同學不需手算這些方法——這正是 g2o、Ceres、GTSAM 這些函式庫要替我們做的事（12.6 節）。

圖 12-3　稀疏性：上萬個節點也能即時求解的祕密



左圖畫出圖 12-2 那個位姿圖的資訊矩陣結構，深色代表非零。可以看到幾乎所有非零元素都擠在對角線附近——因為每條里程計邊只連接相鄰的兩個節點，不會牽涉到其他節點。唯一的例外是右上與左下兩個孤立的方塊，那正是閉環邊造成的，它把第一個與最後一個節點連了起來。

這個 53 個節點的小例子，非零比例已經只有 5.7%；若節點數增加到數千個，非零比例會降到千分之一以下。換句話說，這個矩陣幾乎全是零。

右圖說明這件事為什麼關鍵。一般稠密矩陣的解法運算量是 $O(n^3)$，一千個節點就要做十億次等級的運算，一萬個節點則是一兆次——完全不可能即時。但稀疏解法（稀疏 Cholesky 分解、共軛梯度法等）只處理非零元素，運算量接近 $O(n)$，於是上萬個節點也能在數十毫秒內解完。

這不是細節，而是整個現代 SLAM 得以成立的工程基礎。第 12.6 節的 g2o、Ceres、GTSAM 三套函式庫，核心價值有一大半就在於「把稀疏性用到極致」。同學在第 22 章編譯 ORB-SLAM2 時會安裝 g2o，那時請想起這張圖——它處理的就是這種形狀的矩陣。

稀疏性：大問題能求解的祕密

一個大場景的位姿圖可能有數萬個節點，對應數萬個未知數。直接解這麼大的線性方程組，計算量會大到不可行。幸好 SLAM 問題有一個救命的性質：稀疏性（Sparsity）。

稀疏性的意思是：每一條邊只牽涉兩個節點，而不是全部節點。因此線性化後的大矩陣裡，絕大多數元素是零。利用這個結構，有專門的稀疏矩陣演算法能極快地求解——這正是 EKF-SLAM（第 11 章）辦不到而圖優化能辦到的關鍵：EKF 的共變異數矩陣會變稠密（每個地標都與其他相關），而圖優化保持稀疏。這是圖優化能處理大場景的數學根源。

12.5 Bundle Adjustment

前面的位姿圖主要處理「位姿之間」的約束。在視覺 SLAM 中，還有一個更精細的最佳化問題，同時處理「相機位姿」與「三維地圖點」——這就是光束法平差（Bundle Adjustment,BA）。它是視覺 SLAM 精度的來源，也是第 22、23 章 ORB-SLAM 的核心。

名字的由來

「Bundle」（光束）指的是：從每個三維點，發出許多道光線，分別投射到看見它的各個相機。BA 的任務，就是調整所有相機位姿與所有三維點的位置，讓這些光束盡可能「對齊」——也就是讓每個三維點投影到影像上的位置，盡量接近實際觀測到的像素位置。這個「投影與觀測的差」，稱為重投影誤差（Reprojection Error）。

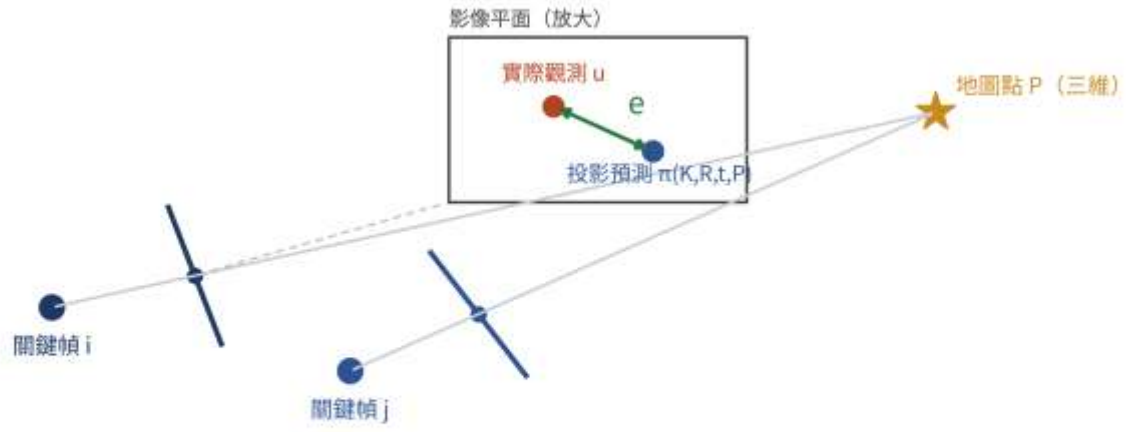
重投影誤差：視覺 SLAM 的殘差

重投影誤差是 BA 的核心殘差，它把第 9 章的相機投影模型用上了：

22. 取一個三維地圖點，用當前估計的相機位姿與內參 K（第 9 章），把它投影到影像上，得到一個「預測像素位置」。
23. 這個地圖點在該影像中，其實有一個「實際觀測到的像素位置」（前端特徵提取得到）。
24. 兩者的差（以像素為單位），就是重投影誤差。

BA 同時調整所有相機位姿與所有三維點，最小化所有重投影誤差的平方和。這是一個巨大的最小平方問題——一個場景可能有數百個相機位姿與數萬個三維點——但同樣靠稀疏性（每個點只被少數相機看到）得以高效求解。

圖 12-4　重投影誤差：Bundle Adjustment 要最小化的那個 e



重投影誤差　$e = u_{observed} - \pi(K, R_i, t_i, P)$

BA 同時調整「所有關鍵幀的位姿」與「所有地圖點的座標」，使全部 e 的平方和最小

Bundle Adjustment 與位姿圖的差別，就在殘差的定義。位姿圖的殘差是「兩個位姿之間的相對關係」，BA 的殘差則是圖中那個綠色箭頭：把三維地圖點 P 用當前估計的相機位姿與內參投影到影像上，投影結果與實際偵測到的特徵點位置之間的差距。

這個定義有兩個重要的性質。其一，它直接以像素為單位——誤差是「差了幾個像素」，

非常直觀，也方便設定門檻（ORB-SLAM 常用的外點判定就是重投影誤差超過某個像素數）。其二，它同時牽涉相機位姿與地圖點座標兩種變數，所以 BA 會一起調整兩者，這正是它名稱中「bundle」的由來——所有光線束一起調。

請把這張圖與第 9 章的圖 9-1 對照著看。那裡的 π 函數是「已知相機位姿與地圖點，算出像素位置」；這裡則是反過來，已知像素位置，去調整相機位姿與地圖點，使算出來的像素位置盡量貼近實際觀測。整個視覺 SLAM 的後端，做的就是這件事。

BA 的變數量遠大於位姿圖——每個地圖點都是三個變數，一個中型場景動輒數萬個點。所幸它的稀疏結構更漂亮：地圖點之間沒有直接的邊，只透過相機相連。利用這個結構做舒爾補（Schur Complement）消元，可以先解出相機位姿再回代地圖點，是 BA 能夠即時運行的關鍵技巧，第 22 章會再提到。

BA 與 Pose Graph 的關係

- Bundle Adjustment：同時最佳化相機位姿與三維點，精度最高，但變數多、計算量大。適合局部（第 22 章的 Local Mapping 執行緒）或關鍵時刻。
- Pose Graph：只最佳化位姿（把三維點「邊緣化」掉），變數少、速度快。適合大範圍的閉環修正（第 22 章的 Loop Closing）。

現代視覺 SLAM 巧妙地結合兩者：平時用局部 BA 維持高精度，閉環時用位姿圖快速拉正大範圍軌跡。這種「局部 BA ＋全域位姿圖」的架構，是 ORB-SLAM 系列的經典設計，第 22、23 章會完整展開。BA 的理論也正是第 2 章 MIT VNAV、CMU 16-833 課程強調的核心——同學現在應能理解，為什麼那些頂尖課程要花大量篇幅在最佳化上。

12.6 g2o、Ceres、GTSAM

前面的數學看似複雜，但好消息是：同學幾乎不需要自己實作這些迭代求解與稀疏矩陣運算。有三個成熟的開源函式庫替我們把重活做好了。認識它們的定位，是進入 SLAM 實作的必備知識。

三大函式庫

- g2o(General Graph Optimization)：專為圖優化設計的 C++ 函式庫，由 SLAM 社群開發。使用方式非常貼近本章的概念——你定義節點（位姿、地標）與邊（約束），它替你求解。ORB-SLAM2 即使用 g2o。對學習者而言，g2o 的架構最能對應本章的位姿圖概念，是理解圖優化的好起點。
- Ceres Solver：由 Google 開發的通用非線性最小平方函式庫，不限於 SLAM。它功能強大、文件完善、自動微分方便（不必手算雅可比），被廣泛用於各種最佳化問題。Cartographer（第 15 章）即使用 Ceres。
- GTSAM(Georgia Tech Smoothing and Mapping)：由喬治亞理工學院開發（第 2 章 CMU 16-833 的 Kaess 教授即為核心貢獻者），以因子圖（Factor Graph）為核心概念，並提供 iSAM2 等增量式求解演算法——能在新資料到來時高效更新，而不必每次從頭重算。

因子圖：另一種等價觀點

GTSAM 使用的「因子圖」是位姿圖的一個更一般化的表述。在因子圖中，待估計的變數是一種節點，而每個約束（稱為「因子」）是另一種節點，連接它所牽涉的變數。它與位姿圖在數學上等價，但因子圖的表述更靈活，能自然地容納各種感測器約束。第 2 章提過，因子圖是 CMU 16-833 課程的核心工具——這也顯示本章內容與國際頂尖課程完全接軌。

函式庫	開發者	核心概念	代表使用者
g2o	SLAM 社群	通用圖優化	ORB-SLAM2
Ceres	Google	通用非線性最小平方	Cartographer
GTSAM	喬治亞理工	因子圖 + 增量式求解	LIO-SAM 等

表 12-1　三大圖優化函式庫比較

對本書讀者的建議是：高中與大學部入門階段，理解「位姿圖—殘差—最小平方」的

概念即可，不必深入函式庫細節；大學部專題與研究階段，則可從 g2o 入手（概念最貼近本章），再依專案需求選用 Ceres 或 GTSAM。第 22、30 章的實作會實際接觸這些工具。

12.7 為什麼現代 SLAM 常用圖優化

本章最後，回答一個貫穿第 11、12 兩章的問題：既然第 11 章的濾波器已經能做 SLAM，為什麼現代主流系統幾乎都改用圖優化？這個對比，是理解當代 SLAM 走向的關鍵。

圖優化的四大優勢

25. 能回頭修正歷史：濾波器把過去壓縮進當前信念，無法回頭；圖優化保留整張圖，閉環時能一次性修正所有歷史位姿，把漂移漂亮地攤平。這是精度差異的最大來源。
26. 反覆線性化，精度更高：EKF 對每個量測只線性化一次（且無法重來）；圖優化在迭代中反覆重新線性化，逐步逼近真解，大幅減少第 11 章提到的線性化誤差。
27. 善用稀疏性，能處理大場景：圖優化的矩陣保持稀疏，配合稀疏演算法可處理數萬節點；EKF-SLAM 的共變異數矩陣會變稠密，隨地標平方成長而卡死。
28. 統一融合多感測器：各種約束都是圖上的邊，里程計、視覺、光達、IMU、GNSS 能自然地緊耦合在同一個問題裡（第 7 章的緊耦合思想）。

圖優化的代價

圖優化並非全無代價。它需要保留較多歷史資料（記憶體較大）、批次求解的單次計算量較大、且實作較複雜。早期受限於運算能力，濾波器才是主流；是計算能力的提升與稀疏求解演算法的成熟，才讓圖優化成為可能並最終勝出。這也提醒同學：技術的主流會隨硬體與演算法演進而變化，理解「為什麼」比記住「現在用什麼」更重要。

比較項目	濾波器（第 11 章）	圖優化（本章）
處理方式	遞迴，逐筆更新	批次，一次性最佳化
歷史修正	無法回頭	可修正所有歷史

比較項目	濾波器（第 11 章）	圖優化（本章）
線性化	每量測一次	迭代中反覆進行
閉環處理	較弱	強項
大場景	EKF 受限（稠密）	善用稀疏，可擴展
計算與記憶體	固定、輕量	較大
代表系統	AMCL、早期 EKF-SLAM	ORB-SLAM、Cartographer

表 12-2　濾波器與圖優化的總比較

至此，第三篇「SLAM 數學基礎」全部完成。同學已經備齊了 SLAM 的完整數學工具箱：第 10 章的座標轉換讓我們能在不同座標系間精確換算、第 11 章的機率與濾波讓我們能處理不確定性、本章的最佳化與圖優化則讓我們能求出最一致的全域解。這三章是抽象的，但它們是後面所有具體系統的共同地基。從下一篇起，我們將把這些數學「用出來」——第四篇從最親民的 2D SLAM 與掃地機器人開始，看 Gmapping 如何用第 11 章的粒子濾波建圖、Cartographer 如何用本章的圖優化做閉環。理論與實作，即將在真實的機器人上會合。

公式或流程

一、SLAM 的最小平方目標函數

圖優化 SLAM 的核心，是求一組狀態 X（所有位姿與地標），使加權殘差平方和最小：

$$X^* = \operatorname{argmin} \Sigma_k \; e_k(X)^T \cdot \Omega_k \cdot e_k(X)$$

其中 e_k 是第 k 條邊的殘差（量測與預測的差）、Ω_k 是該邊的資訊矩陣（可信度權重，即共變異數的反矩陣）、X* 是最佳解。這條式子把整個 SLAM 濃縮成一句話：找一組位姿與地圖，使所有約束的加權矛盾最小。第 11 章的機率最大化，在高斯假設下正等價於這個最小平方。

二、殘差的一般形式

位姿圖中，一條連接節點 i 與 j 的邊，其殘差為：

$$e_{ij} = 量測的相對位姿\ (z_{ij}) \ominus 預測的相對位姿\ (x_i, x_j)$$

符號 ⊖ 表示在位姿空間（第 10 章）中做「相減」——由於位姿含旋轉，這個相減不是普通減法，而需在旋轉群上進行（第 2 章 MIT VNAV 的流形運算）。殘差為零表示約束完美滿足。

三、重投影誤差（Bundle Adjustment）

視覺 SLAM 中，三維點 P 在第 i 個相機影像上的重投影誤差為：

$$e = u_observed - \pi(K, R_i, t_i, P)$$

其中 u_observed 是實際觀測到的像素座標、π(·) 是第 9 章的投影函數（用相機位姿 R_i、t_i 與內參 K 把三維點 P 投影到影像）。BA 同時對所有相機位姿與三維點最小化所有重投影誤差平方和。

四、迭代求解流程（高斯—牛頓 / LM）

圖 12-5　圖優化的迭代求解流程

① 初始化：用里程計串起粗略的位姿（初始猜測 X_0）

↓

② 線性化：在當前 X 附近計算所有殘差與雅可比矩陣

↓

③ 建立線性系統：$H \cdot \Delta x = -b$　（H 為稀疏資訊矩陣）

↓

④ 稀疏求解：利用稀疏性快速解出修正量 Δx

↓

⑤ 更新：$X \leftarrow X \oplus \Delta x$　（在位姿空間上更新）

↓

⑥ 收斂？否 → 回到 ②　/　是 → 輸出最佳解 X*

這整個流程，就是 g2o、Ceres、GTSAM 在背後替我們執行的。同學理解流程即可，不需手動實作——正如開車不必自己造引擎。

12.8　圖優化的當代發展：更快、更大、更不怕錯

本章介紹的框架自 2010 年前後定型以來，骨架幾乎沒有改變。但在三個方向上，它一直在往前推進——速度、規模，以及對錯誤資料的容忍度。

一、增量式求解：不必每次都重算整張圖

最直接的問題是：機器人每走一步就多一個節點，難道每次都要重解整張圖？第 12.6 節提到的 GTSAM 給出的答案是 iSAM2——它把因子圖整理成一棵貝氏樹，新加入一條邊時只需要重新計算受影響的那一小部分分支，其餘維持不變。

這讓最佳化從「批次處理」變成「可即時增量更新」，是圖優化能在機器人上線的關鍵。它也解釋了一個現象：平常建圖時 CPU 很輕鬆，一偵測到閉環就突然忙碌起來——因為閉環邊會牽動整棵樹。

二、全域式方法重新回到檯面

在離線重建這一側，近年出現了值得注意的翻轉。長期以來，增量式的重建（逐張加入影像、反覆做 BA）被認為精度最高但極慢；全域式方法快但不夠穩。2024 年 ECCV 的一項工作重新設計了全域流程——視圖圖校正、全域定位、結構精修三個步驟，其中全域定位階段同時估計相機位置與影像點。

成果相當驚人：在一組三萬六千張影像、相機跨越兩百五十公尺以上的大型資料集上，它在 5.5 小時內完成重建，一公尺內的召回率達九成；而傳統增量式方法跑超過七天，召回率僅約五成。精度與增量式相當或更好，速度卻快了數個數量級。

對同學的意義是：如果專題需要用數千張照片重建一個校園或一棟建築（第 17 章、第 31 章的專題常有這類需求），現在有了實際可行的工具，不必再等上好幾天。

三、強健核函數：當前端給了錯誤的閉環

圖優化有一個致命弱點，本章 12.3 節的「限制條件的一致性」已經提過：它假設每一條邊都是對的。但前端會出錯——第 20 章的視覺詞袋可能把兩個長得很像的走廊誤判為同一處，第 17 章的點雲配準可能在對稱結構中收斂到錯誤的解。

一條錯誤的閉環邊有多嚴重？它會像一條強力的橡皮筋，把兩段本來不相鄰的軌跡硬拉在一起，整張地圖當場扭曲變形。而且因為最小平方對大殘差的懲罰是平方成長，這條錯誤的邊反而擁有最大的話語權——這是最小平方法天生的脆弱之處。

對策有兩個層次。第一層是強健核函數：把殘差的平方懲罰換成 Huber 或 Cauchy 這類成長較慢的函數，使大殘差的影響被壓住，離群值不再主導整個解。第二層更徹底——讓最佳化自己決定要不要相信某條邊，例如為每條閉環邊加一個開關變數，若它與其他約束嚴重

衝突就自動被關掉。

實務上這件事極為重要。同學在第 29 章做實體建圖時，若發現「地圖本來好好的，突然整個扭曲」，第一個該懷疑的就是誤判的閉環。而把求解器的強健核函數打開，往往就能解決。

四、與學習方法的結合

最後值得一提的，是圖優化與深度學習的分工正在定型。目前最成熟的模式不是用網路取代最佳化，而是讓網路負責前端——偵測與匹配特徵、判斷閉環、估計深度先驗——再把結果交給傳統的因子圖後端求解。

這個分工的理由與第 7.8 節、第 9.8 節、第 11.8 節完全一致：後端的幾何與機率結構提供可解釋性、不確定度與數學保證，前端的感知部分則交給擅長從資料中學規律的模型。第 23 章的 ORB-SLAM3 與第 32 章的 AI 輔助導航，都可以放在這個框架下理解。

> **一個實用的除錯順序**
>
> 地圖扭曲 → 先懷疑錯誤的閉環邊（打開強健核函數，或提高閉環的驗證門檻）。
>
> 地圖整體歪斜但局部正常 → 懷疑里程計的系統性偏差（如圖 12-2 左圖），需要校正輪徑或輪距（第 29 章）。
>
> 閉環偵測不到 → 問題在前端，不在後端（第 15 章的分支定界、第 20 章的視覺詞袋）。
>
> 求解很慢 → 檢查是否誤用了稠密解法，或節點數是否該做關鍵幀篩選。

案例

案例 12-1　閉環的威力：一張歪掉的地圖如何被拉正

這是圖優化最戲劇性的展示，幾乎每個 SLAM 課堂都會演示。一台機器人繞行一棟大樓一圈，全程只靠里程計與掃描匹配建圖。走到一半，地圖看起來還算正常；但快繞完一圈時，

由於漂移累積，地圖開始「歪斜」——本該閉合的走廊，終點與起點錯開了好幾公尺，同一面牆甚至被畫成兩道分岔的線。

然後，閉環偵測啟動：系統認出「我現在看到的，就是一開始的大廳」，產生一條閉環邊。圖優化隨即執行——在螢幕上，同學會看到整張歪掉的地圖「唰」地一下被拉正：分岔的牆合而為一、錯開的走廊接合、整個軌跡變得規整。這一瞬間，本章 12.2 節「把矛盾均攤到整條軌跡」的抽象描述，變成了眼前活生生的畫面。

這個案例的震撼力在於：它讓同學親眼看到「一條邊如何修正整張圖」。這是濾波器做不到的——濾波器沒有保留歷史位姿，無法回頭。圖優化的全部價值，在這一個「拉正」的瞬間展露無遺。第 15 章 Cartographer 實作時，同學將親手觸發這個時刻。

案例 12-2　GTSAM 的血脈：從課堂到全球產業

第 2 章介紹 CMU 16-833 課程時提過：Michael Kaess 教授與 Frank Dellaert 教授開發的 GTSAM 函式庫，把因子圖與增量式平滑的理論包裝成開源工具，如今已是全球機器人公司與研究室的標準配備。本章的內容，正是理解 GTSAM 在做什麼的鑰匙。

GTSAM 的一項關鍵創新是 iSAM2（增量式平滑與建圖）。傳統圖優化每次有新資料都要重解整個問題；iSAM2 則聰明地只更新「受新資料影響的那部分」，大幅提升效率，讓大規模、長時間的 SLAM 得以即時運行。這解決了本章 12.7 節提到的「圖優化計算量大」的痛點。

這個案例串起了本書的多條線索：第 2 章的名校課程、本章的圖優化數學、以及真實世界的產業應用。它告訴同學：你正在學的這些數學，不是象牙塔裡的玩具，而是驅動水下機器人、自駕車、AR 裝置的實際引擎。理解本章，你就站在了與那些頂尖工程師相同的知識基礎上。

案例 12-3　從測量學到 SLAM:Bundle Adjustment 的百年血統

Bundle Adjustment 常被誤以為是電腦視覺的新發明，其實它的血統可追溯到十九世紀的攝影測量學（Photogrammetry）。當年的測量學家用多張空拍照片重建地形，面對的正是同一個問題：如何調整相機位置與地面點，讓所有照片的投影一致？他們發展出的平差理論，正是今天 BA 的前身。

這段歷史給同學兩個啟示。其一，最小平方與平差的思想歷久彌新——從十九世紀的測繪、二十世紀的阿波羅導航（第 10、11 章），到二十一世紀的 SLAM，人類處理「多個帶誤差量測」的核心方法一脈相承。其二，跨領域的知識常常相通：測量學、攝影測量、電腦視覺、機器人學，在 BA 這個點上交會。這也呼應第 10 章「機械手臂與行動機器人共用同一套數學」的觀察——數學是所有工程領域的共同語言，學好基礎，處處相通。

實作活動

實作 12-1　位姿圖最佳化動手做

目的：透過紙上推演與程式實作，親身理解位姿圖如何被「拉正」，體會閉環約束的威力。本實作分紙上、程式兩部分，適合不同深度的教學。

第一部分：紙上位姿圖（全體）

16. 在方格紙上畫一個「正方形巡行」：機器人從原點出發，依里程計「向右 4 格、向上 4 格、向左 4 格、向下 4 格」應回到原點。先照里程計畫出四個轉角位姿。
17. 加入漂移：假設每段的轉角都少轉了一點（例如每次只轉 85 度而非 90 度），重新畫出軌跡，觀察終點如何無法回到原點（出現漂移缺口）。
18. 加入閉環邊：宣告「終點就是起點」。用直覺思考：要讓終點回到起點，又盡量不破壞四條邊的長度，四個轉角應該如何微調？動手把「拉正後」的軌跡畫出來。
19. 討論：閉環邊把誤差「均攤」到四個轉角，而不是集中修正某一個。為什麼均攤比集中好？

第二部分：Python 位姿圖最佳化（大學部）

20. 用 Python 建立一個簡單的 2D 位姿圖：數個位姿節點、連接它們的里程計邊（含模擬漂移）、以及一條閉環邊。
21. 實作殘差計算：對每條邊，算出「量測的相對位姿」與「當前估計的相對位姿」之差。
22. 用現成的最佳化工具（如 scipy.optimize.least_squares，或安裝 g2o / GTSAM 的 Python 介面）求解，最小化所有殘差平方和。
23. 把「最佳化前」與「最佳化後」的軌跡畫在一起，觀察漂移的軌跡如何被閉環邊拉正回閉合。
24. 實驗資訊矩陣：把閉環邊的權重（資訊矩陣）分別調大與調小，觀察拉正的力道如何變化。權重大時，系統更「相信」閉環，拉得更用力。

第三部分：觀察真實 SLAM 的閉環（有 ROS 環境者）

25. 用 Cartographer（第 15 章）或其他支援閉環的 SLAM，在模擬環境中讓機器人繞一圈，在 RViz 中即時觀察：繞行時地圖逐漸漂移，閉環觸發瞬間地圖被拉正。
26. 錄下這個「拉正瞬間」，對照案例 12-1 與本章 12.2 節，指認哪些是里程計邊、哪條是閉環邊。

討論問題

12. 第一部分中，加入閉環邊後，你如何微調四個轉角？最佳化把誤差均攤，和「把所有誤差算在最後一段」相比，哪個結果的地圖更合理？為什麼？
13. 第二部分中，調大閉環邊的權重，拉正的力道有何變化？若閉環偵測其實是「誤判」（錯誤的閉環），高權重會造成什麼後果？
14. 本章說圖優化「能回頭修正歷史」，而第 11 章的濾波器不能。從你的實驗結果，說明這個差異如何影響最終地圖品質。

15. （大學部）為什麼稀疏性讓大型位姿圖得以快速求解？若每條邊都牽涉所有節點（不稀疏），會發生什麼？

延伸挑戰（大學部）

安裝 g2o 或 GTSAM 的 Python 綁定，載入一個公開的標準位姿圖資料集（如 g2o 格式的 intel、manhattan 資料集），執行最佳化並用 matplotlib 視覺化最佳化前後的差異。進一步嘗試「加入一條錯誤的閉環邊」，觀察它如何破壞整張地圖——這將幫助你理解為何第 20、22 章要花心力在「穩健的閉環偵測」上，以及穩健核函數（Robust Kernel）為何重要。

延伸閱讀

15. G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010.（圖優化 SLAM 最經典的入門教程，本章主要依據）
16. 高翔、張濤，《視覺 SLAM 十四講》第 6 講（非線性最佳化）、第 9-10 講（後端與 BA、位姿圖）.（華文世界最清楚的圖優化實作教材）
17. R. Kümmerle et al., "g2o: A General Framework for Graph Optimization," ICRA, 2011.（g2o 函式庫原始論文）
18. F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," Foundations and Trends in Robotics, 2017.（GTSAM 與因子圖的權威著作，可免費取得）
19. B. Triggs et al., "Bundle Adjustment — A Modern Synthesis," Vision Algorithms, 2000.（Bundle Adjustment 的權威綜述）
20. Ceres Solver 官方文件（ceres-solver.org）與 GTSAM 官方教學（gtsam.org）.（三大函式庫的第一手實作指引）
21. CMU 16-833 Robot Localization and Mapping 課程（第 2 章介紹）之因子圖與圖優化單元.（研究所級的圖優化教材）

第三篇完結：從數學回到機器人

第三篇三章到此結束。第 10 章給了表達位姿的語言，第 11 章給了描述不確定性的語言，本章給了求解的方法。三者合起來，就是整個現代 SLAM 的數學骨架。

回頭看，這三章其實在回答同一個問題的三個層面。「我在哪裡？」——第 10 章說，這是一個位姿 T；第 11 章說，它其實是一個分布；本章說，要找出它，就把所有約束放在一起解一個最小平方問題。

從下一章開始，第四篇要把這些工具真正用起來。第 13 章會把光達的掃描資料變成一張佔據格地圖——那裡會同時用到第 10 章的座標轉換、第 11 章的機率更新，以及本章「用多次觀測求一致解」的思想。第 14 章的 Gmapping 是粒子濾波的實作，第 15 章的 Cartographer 則是本章位姿圖的實作。

換句話說，第三篇學的不是「數學」，而是後面六篇每一章都會用到的工具。如果某個概念現在還覺得抽象，不必焦慮——到第 29 章親手看著一張歪掉的地圖在閉環那一刻被拉正時，本章的一切都會變得具體。

小結

8. 現代 SLAM 採前端 + 後端架構：前端把感測資料轉成幾何約束（掃描匹配、特徵匹配、閉環偵測），後端在所有（常互相矛盾的）約束下求最一致解；圖優化屬於後端。
9. 位姿圖以節點（待估的位姿與地標）與邊（相對量測約束）描述 SLAM;里程計邊像鏈子串起位姿、閉環邊形成迴路，是消除累積漂移的關鍵。
10. 每條邊有殘差（量測減預測）與資訊矩陣（可信度權重，即共變異數的反矩陣）；圖優化把各種感測器約束統一成帶權重的邊，實現多感測器緊耦合。

11. SLAM 化為最小平方問題：找一組狀態使加權殘差平方和最小；用平方兼具數學（可微）與機率（高斯下等價於最大後驗）理由，以高斯—牛頓 / LM 迭代求解，並靠稀疏性處理大場景。

12. Bundle Adjustment 同時最佳化相機位姿與三維點，最小化重投影誤差，是視覺 SLAM 精度的來源；現代系統結合「局部 BA（高精度）+ 全域位姿圖（快速閉環）」。

13. g2o（通用圖優化，ORB-SLAM2 使用）、Ceres（Google 通用最小平方，Cartographer 使用）、GTSAM（因子圖 + 增量式 iSAM2）三大函式庫替我們完成求解；因子圖是位姿圖的一般化表述。

14. 現代 SLAM 多採圖優化，因它能回頭修正歷史、反覆線性化提升精度、善用稀疏性處理大場景、統一融合多感測器；代價是記憶體與計算量較大，是硬體與演算法進步後才成為主流。

第三篇「SLAM 數學基礎」到此完成。我們已備齊完整的數學工具箱：座標轉換（第 10 章）、機率與濾波（第 11 章）、最佳化與圖優化（本章）。這三章抽象卻是一切的地基。從下一篇起，理論將在真實機器人上落地——第四篇從最親民的 2D SLAM 與掃地機器人開始，你會看到 Gmapping 如何運用第 11 章的粒子濾波、Cartographer 如何運用本章的圖優化。數學與實作，即將會合。